



哈爾濱工業大學(深圳)

HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

Asterinas EXT4设计开发文档

面向RustOS的高性能强一致性EXT4文件系统

项目成员：

姓名	年级	联系方式 (QQ)
俞杰	大三	2113200249
梁丙煜	大三	1987247793
王毅航	大三	3223909812

指导教师：夏文、李诗逸

2026年08月

目 录

摘要	1
1 项目介绍	3
1.1 项目背景及意义	3
1.2 项目目标	4
1.3 项目完成情况	4
1.4 国内外研究概况	7
2 系统总体设计	11
2.1 设计目标与总体原则	11
2.2 原生EXT4总体架构	12
2.3 代码模块划分	13
2.4 数据路径	15
2.5 元数据事务路径	16
2.6 挂载与恢复路径	16
3 关键模块设计与实现	17
3.1 EXT4磁盘结构与特性支持	17
3.2 Extent树与空间管理	20
3.3 EsCache与NodeCache	24
3.4 JBD2日志子系统	25
3.5 Buffered I/O与PageCache	32
3.6 O_DIRECT与缓存一致性	34
3.7 mmap基础路径与一致性边界	37
3.8 并发、锁序与错误处理	38
4 性能优化技术研究	42
4.1 性能评估口径与归因方法	42
4.2 关键瓶颈与优化路线	43
4.3 Extent与空间分配优化	44
4.4 缓存与I/O路径优化	44
4.5 fio性能评估	45
4.6 外部电源保护模式性能评估	46
4.7 SQLite真实负载评估	48
4.8 RustOS文件系统优化方法总结	48
5 系统测试与验证	49
5.1 测试环境与结果	49
5.2 测试体系总体设计	49
5.3 xfstests兼容性评估	50

5.4	并发与缓存一致性验证	51
5.5	崩溃矩阵与恢复验证	51
5.6	数据持久化Oracle与独立检查体系	52
5.7	Linux双向互操作验证	53
5.8	测试结论与验证边界	54
6	项目创新点	54
7	开发过程与工程管理	56
7.1	开发路线	56
7.2	代码组织与工程约束	57
7.3	典型问题与处理	58
7.4	代码迭代与测试记录	59
8	AI使用说明	59
8.1	披露范围	59
8.2	AI工具与模型清单	59
8.3	使用场景	60
8.4	人机分工	60
8.5	交互记录与可追溯性	60
8.6	诚信声明	60
9	总结	62
附录 A	核心实现细节补充	64
附录 B	复现命令	67
参考资料		70

摘 要

本项目面向2026年全国大学生计算机系统能力大赛操作系统设计赛，在Safe Rust操作系统Asterinas中设计并实现原生EXT4文件系统。系统支持POSIX核心接口、Extent块管理和JBD2日志，可以读写标准EXT4磁盘格式，并支持fio、SQLite等应用运行。

在完成EXT4基本读写功能的基础上，本项目进一步解决了文件系统在实际运行中必须面对的几个关键问题，包括应用兼容、崩溃恢复、并发访问、缓存一致性和I/O性能。系统实现了Buffered I/O、基础mmap、fsync/fdatasync、O_DIRECT以及并发读写，并建立了完整的JBD2事务提交、检查点和挂载恢复流程。在此基础上，我们针对Extent管理、缓存访问和日志提交等性能瓶颈进行了优化。针对配有UPS或备用电源的受控环境，还设计了一种可选的外部电源保护模式：正常运行时尽量减少日志和元数据的同步写盘，在收到掉电通知后停止新的文件系统操作，回滚尚未完成的事务，再将已经完成的修改统一写回磁盘，从而在保证一致性的同时降低频繁持久化带来的性能开销。

本项目主要围绕以下四个目标展开：

1. 完成Asterinas EXT4主体功能,实现POSIX核心接口、目录操作、Extent管理、inode与块分配，以及VFS和块设备适配。
2. 实现JBD2日志与崩溃恢复，支持transaction、handle、commit、checkpoint和recovery，并维护ordered模式下的数据写回与日志提交顺序。
3. 完成与Linux EXT4的性能对比，使用fio、filebench或其他测试工具评估文件系统基本性能，使用SQLite等应用测试真实工作负载。
4. 完成性能创新优化，围绕Rust语言特性、Asterinas平台边界以及实际测试中暴露的性能瓶颈开展优化。

在正确性验证方面，项目采用了从功能测试、崩溃恢复到磁盘格式检查的多层验证方法。xfstests目前共有78项测试通过，其中O_DIRECT专项10项全部通过。针对最关键的崩溃一致性问题，我们分别在标准日志和4 MiB小日志环境下进行了逐崩溃点测试，共覆盖2560个崩溃点，恢复后均通过一致性检查。此外，数据持久化oracle覆盖232个工作负载和308条断言，walcheck检查了2360个已提交元数据版本的写回顺序，均未发现异常。结合accounting、checksum以及Linux双向互操作测试，进一步验证了文件系统在事务顺序、磁盘结构和EXT4格式兼容性方面的正确性。

在性能方面，fio顺序直接I/O测试覆盖4 KiB至1 MiB的五种块大小。顺序读吞吐率为Linux EXT4的110.0%–113.4%，顺序写吞吐率为105.1%–110.4%。同文件并发写测试中，numjobs=2和numjobs=4时的吞吐率分别为1380 MB/s和2446 MB/s，分别为Linux

EXT4的103%和111%。SQLite speedtest1耗时由初赛阶段的234.9 s缩短至86.2 s，下降63.3%。在外部电源保护模式下，高频fsync顺序写吞吐率达到标准JBD2模式的180.59%–254.81%，对应性能提升80.59%–154.81%。

编号	目标	完成情况	决赛成果
1	Asterinas EXT4主体功能	目标范围内已完成（100%）	POSIX核心接口、Extent、目录、VFS、PageCache、O_DIRECT和基础mmap均已实现并验证。
2	JBD2与崩溃恢复	目标范围内已完成（100%）	原生事务生命周期与恢复闭环完成，两类日志矩阵共2560个崩溃点0 red。
3	Linux EXT4性能对比	目标范围内已完成（100%）	完成fio顺序、并发及SQLite同口径对照，正式结果均绑定完整性检查。
4	性能创新优化	目标范围内已完成（100%）	完成原地Extent、EsCache、NodeCache、写回与O_DIRECT一致性、并发优化及带事务回滚的掉电保护模式。

项目主要目标完成情况

综上，本项目已经完成预定的四项主要目标。系统不仅实现了Asterinas上的原生EXT4主体功能和完整的JBD2事务恢复机制，还能够与Linux EXT4进行磁盘镜像的双向读写。在此基础上，我们进一步完成了针对Extent、缓存、并发和日志路径的性能优化。上述功能均通过xfstests、崩溃恢复测试、数据持久化检查以及Linux互操作测试进行了验证。

1 项目介绍

1.1 项目背景及意义

文件系统是操作系统中负责管理和持久化数据的核心模块。对数据库、消息队列、编译构建等应用来说，仅仅能够完成文件的创建、读取和写入是不够的，还需要保证fsync、rename、truncate等操作在系统异常或重新启动后仍能得到正确、可预期的结果。同时，现代文件系统中，同一个文件可能通过PageCache、mmap、O_DIRECT等多条路径访问，文件增长和修改还会涉及Extent、空间分配以及日志事务等一系列元数据变化。因此，一个真正可用的文件系统不仅要“能读能写”，还需要解决缓存一致性、并发访问和崩溃恢复等问题。

目前成熟的文件系统大多由C语言实现。随着功能不断增加，文件系统内部存在大量共享状态、指针和并发操作，代码复杂度也随之提高。Rust提供的所有权、类型检查和错误处理机制，可以在一定程度上降低内存越界、悬垂引用等问题带来的风险。Asterinas是一个使用Rust开发并兼容Linux ABI的操作系统，其framekernel架构为安全的内核开发提供了新的实现方式。但与此同时，Asterinas的VFS、PageCache、虚拟内存和块设备接口与Linux内核并不相同，Linux中成熟的EXT4和JBD2实现无法直接移植，需要结合Asterinas的系统架构重新设计和实现。

EXT4是Linux中长期使用的主流文件系统，具有成熟稳定的磁盘格式、Extent块管理机制、JBD2日志以及完善的e2fsprogs工具链，在数据库、服务器和普通计算机中都有广泛应用。更重要的是，采用标准EXT4格式后，可以直接使用Linux创建和检查磁盘镜像，也能够以Linux EXT4作为功能和性能上的参考。因此，本项目选择在Asterinas内核中实现原生EXT4，而不是重新设计一种简化的文件系统格式，也不是简单地将现有用户态EXT4库封装到内核中。

本项目希望解决的核心问题，是如何在RustOS的系统架构下实现一个真正可用的EXT4文件系统。在功能上，我们实现标准EXT4磁盘镜像的读写和常用POSIX文件操作；在可靠性上，引入JBD2事务和崩溃恢复机制，保证异常情况下文件系统仍能恢复到一致状态；在性能上，围绕Extent、PageCache、O_DIRECT、日志提交和并发访问等关键路径进行优化；在验证上，通过xfstests、崩溃恢复测试、数据持久化检查以及Linux双向互操作，对文件系统的功能、正确性和兼容性进行系统验证。通过这些工作，我们希望探索一套适用于Asterinas等RustOS的成熟文件系统实现与优化方法。

1.2 项目目标

项目目标由功能、持久化、一致性、兼容性和性能五个方面组成。每项目标都对应明确的实现内容和验证方法，完成情况结合功能测试、崩溃恢复、互操作与性能结果综合判断。

目标	设计内容	验收方式
原生EXT4主体功能	在fs_impls/ext4中实现VFS适配、inode、目录、Extent、块组、分配和磁盘特性。	xfstests以及Linux创建镜像的挂载读写。
JBD2崩溃一致性	实现Handle、credits、Transaction、group commit、revoke、lazy checkpoint、backpressure、abort与recovery。	标准与小日志崩溃矩阵、e2fsck、walcheck和数据持久化oracle。
多路径数据一致性	Buffered I/O接入PageCache，O_DIRECT建立范围排空与失效协议。	缓存交叉读写、O_DIRECT专项、并发hash与mmap基础路径测试。
格式和接口兼容性	支持主要EXT4特性、POSIX核心接口及Linux双向磁盘互操作。	xfstests、Linux mount/read/write与宿主e2fsck。
性能与方法研究	通过分层profiling优化Extent、空间分配、BIO、readahead、NodeCache、EsCache和writeback。	fio、SQLite同口径对照，并以完整性检查。

表1-1 项目目标与验收方式

1.3 项目完成情况

项目已经完成表1-1所列五项目标。完成度以表中规定的功能范围和验收方式为准：既检查功能是否存在，也检查其是否通过标准测试、崩溃验证、Linux互操作或性能对照。

目标	完成情况	已实现能力	验收证据
原生EXT4主体功能	目标范围内已完成（100%）	完成原生VFS适配、inode与目录操作、Extent原地编辑、块组和空间分配、特性检查及标准磁盘格式读写。	xfstests有78项通过；Linux创建镜像可直接挂载读写。
JBD2崩溃一致性	目标范围内已完成（100%）	完成handle与credit、事务切换、group commit、ordered data、revoke、barrier、lazy checkpoint、backpressure、abort和三阶段恢复。	标准日志931个崩溃点和小日志1629个崩溃点均通过一致性检查；walcheck完成2360个已提交元数据新版本的写序核对。
多路径数据一致性	目标范围内已完成（100%）	Buffered I/O接入PageCache；fsync与fdatasync按事务号等待；O_DIRECT实现范围排空、失效和unwritten-first协议。	O_DIRECT专项10项全部通过；数据oracle覆盖232个工作负载、308条断言；缓存交叉访问与并发hash检查通过。
格式和接口兼容性	目标范围内已完成（100%）	完成赛题范围内的POSIX核心接口、主要EXT4盘面特性、metadata_csum与64bit格式处理，并支持Linux双向磁盘互操作。	xfstests、宿主e2fsck、checksum/accounting检查以及Linux mount/read/write共同验证。

续下页

续表1-2

目标	完成情况	已实现能力	验收证据
性能与方法研究	目标范围内已完成（100%）	完成分层归因、原地Extent、EsCache、NodeCache、连续写回、直接I/O、同文件并发路径及带事务筛选与回滚的掉电保护模式优化。	顺序读写达到Linux的105.1%–113.4%；并发写达到Linux的103%和111%；SQLite耗时由234.9 s降至86.2 s；外部电源保护模式下高频fsync写性能提升80.59%–154.81%。

表1-2 项目目标完成情况

为了使完成情况能够直接回到代码和测试，表1-3进一步列出实现位置、验收依据和适用边界。

实现项	代码落点	决赛验收依据
文件与目录操作	impl_for_vfs/、inode/、inode/dir/	xfstests结果和Linux镜像读写。
Extent与空间管理	inode/extent_manager/、block_group.rs、fs.rs	原地编辑检查、fallocate、空间压力用例和崩溃矩阵。
JBD2与挂载恢复	journal/	标准日志931点、小日志1629点均0 red，walcheck为0 violation。
校验和与磁盘特性	feature.rs、checksum.rs	checksum/accounting检查、Linux字节对照和宿主e2fsck。
Buffered I/O与同步	inode/mod.rs、PageCache后端	缓存交叉读写、fsync/fdatasync用例、数据oracle和SQLite完整性检查。
O_DIRECT	inode/mod.rs、Bio与IoBatch接口	专项xfstests 10项通过，fio五档顺序读写及同文件并发对照完成。

续下页

续表1-3

实现项	代码落点	决赛验收依据
基础mmap	Vmo与PageCache映射路径	基础映射读写、共享页访问和普通同步路径测试。
性能优化	Extent、缓存、writeback、block接口和journal/mod.rs	并发写达到Linux的103%和111%，SQLite耗时降低63.3%，外部电源保护模式下高频fsync写性能提升80.59%–154.81%。
系统化验证	test/与崩溃验证工具链	xfstests、数据oracle、多项独立检查和Linux互操作。

表1-3 项目实施位置与验收依据

五项目标均已达到预定验收要求。与初赛版本相比，决赛版本进一步闭合了原生JBD2生命周期、系统化崩溃验证、Linux双向互操作和性能优化证据链。

1.4 国内外研究概况

EXT4是Linux生态中长期使用的主流本地文件系统。它并不是从零设计的新文件系统，而是在EXT3已有磁盘格式、日志机制和工具链基础上逐步演进形成的。早期EXT4研究工作的主要目标，是在保持较好兼容性和可靠性的同时，解决EXT3在大容量磁盘、连续空间管理和大文件访问方面的扩展性问题。为此，EXT4引入了Extent映射、48位块号、更大的文件系统容量、持久化预分配和更灵活的块组组织方式 [1]。

从工程实现角度看，Linux EXT4的性能并不是由某一个局部函数决定的，而是由多种机制共同构成。Extent使用一个映射项描述连续逻辑块到连续物理块的关系，降低大文件和顺序访问中的映射元数据开销；multi-block allocator (mballoc) 倾向于一次分配较大的连续区域；delayed allocation (delalloc) 则将物理块分配推迟到脏页写回阶段，使文件系统能够综合多个页面的写入需求后再选择物理位置 [3, 2]。这些机制与PageCache、writeback、预读、块设备调度和日志提交相互配合，共同决定普通buffered I/O和数据库负载中的实际性能。

Linux VFS为不同文件系统提供了统一的系统调用与内核对象接口。open、read、write、stat和mmap等用户接口首先进入VFS，随后再通过inode、file、dentry和superblock等对象分派到具体文件系统实现 [4]。因此，一个文件系统即使能够正确解析EXT4磁盘

格式，也不能直接等同于完整的内核文件系统。它还必须适配目标操作系统中的路径解析、文件对象生命周期、页缓存、内存映射、并发锁和块设备接口。这也是用户态EXT4库与内核态EXT4实现之间的重要区别。

在崩溃一致性方面，Linux EXT4主要依赖JBD2组织元数据事务。其默认的ordered模式只将文件系统元数据写入日志，但要求相关普通数据块在对应元数据提交之前先写入home block（文件数据或元数据在文件系统上的原始目标块）。系统恢复时根据完整的commit记录判断事务是否已经提交，并对已经提交但尚未完成checkpoint的元数据进行重放 [2, 5]。这种设计避免了将所有普通文件数据写入日志所产生的高额写放大，同时又比完全不约束数据顺序的writeback模式提供更强的恢复语义。

然而，日志在提供崩溃恢复能力的同时，也是以一定的运行性能为代价换取一致性保证，其本身正是一致性与性能之间矛盾的集中体现，而非这一矛盾的消解。事务提交需要descriptor、metadata payload、commit block以及设备flush按照规定顺序到达稳定存储；频繁fsync会放大日志提交和设备屏障成本；延迟分配虽然能够减少小粒度元数据更新，但又要求系统正确维护脏页、空间预留、写回失败和ENOSPC语义。写时复制文件系统可以避免原地覆盖，但通常需要承担额外的空间占用、树结构更新和垃圾回收成本。可见，现代文件系统的各类一致性机制并未真正消除上述矛盾，而只是在持久化语义、写放大、缓存复杂度和运行性能之间，选择了不同的权衡方式。

已有研究也表明，成熟文件系统仍需要持续处理大量语义错误、并发错误和异常路径问题。针对Linux文件系统长期演进过程的研究发现，文件系统补丁中存在大量维护、错误修复、性能和可靠性改进，其中语义错误、并发同步和失败处理路径尤其难以完全消除 [6]。这说明实现一个可运行的文件系统只是第一步，长期可靠性还依赖系统化测试、故障注入、边界检查和回归验证。

随着Rust在系统软件中的应用不断增加，研究者开始探索如何利用所有权、生命周期和类型系统改善操作系统及文件系统的安全性和可维护性。Theseus尝试使用Rust的语言级机制表达操作系统组件边界和状态关系，将部分系统不变量交由编译器检查 [11]。Bento则提出了面向Linux内核文件系统的安全Rust开发框架，希望在保持较低运行开销的同时改善文件系统开发、调试和升级效率 [9]。

在崩溃一致性方向，SquirrelFS进一步使用Rust typestate表达持久化内存文件系统中的元数据更新顺序，将部分崩溃一致性约束转化为编译期类型检查 [10]。这些研究说明，Rust的价值不仅体现在减少悬空指针、越界访问和重复释放等内存安全问题，也可以用于表达对象状态、操作阶段和更新顺序。但是，类型系统能够保证哪些性质，仍然取决于实现者是否将相应的不变量正确编码到接口和类型中。

因此，Rust并不会自动提高文件系统性能，也不会自动解决崩溃一致性问题。JBD2

的事务边界、commit block前的持久化顺序、revoke语义、fsync与PageCache的协同，以及O_DIRECT与buffered I/O的缓存可见性，本质上仍然属于文件系统协议和系统架构问题。Rust提供的是更加清晰的所有权边界、错误传播方式和状态表达能力，真正的性能与一致性仍需要针对I/O路径、缓存、锁和日志机制进行设计。

Asterinas为本项目提供了与Linux内核不同的运行环境。Asterinas采用framekernel架构，并通过OSTD提供面向安全Rust内核开发的基础设施，同时保持对Linux ABI的兼容 [12, 13]。对于EXT4而言，这意味着系统不能直接复用Linux中成熟的address_space、writeback、JBD2和block layer实现，而需要重新适配Asterinas的VFS、PageCache、Vmo、bio、同步原语和virtio-blk设备路径。

开源ext4_rs提供了一个使用Rust表达EXT4磁盘结构与基础文件操作的参考实现 [14]，对项目前期理解磁盘字段和Rust接口组织具有参考价值。但它定位为与具体操作系统相对独立的EXT4库，不包含Asterinas所需的 VFS、PageCache、BIO、并发控制和完整JBD2。当前实现位于 `kernel/src/fs/fs_impls/ext4/`，由原生模块直接承担磁盘语义和内核运行时，不以ext4_rs作为运行时核心，也不是外部库与内核包装层的组合。

表1-4对主要研究与工程对象进行了总结。

研究或工程对象	主要特点	本项目的借鉴或区别
Linux EXT4	成熟的Linux本地文件系统，具有Extent、JBD2、delalloc、mballoc、PageCache和writeback等完整机制。	作为盘面语义、JBD2协议和性能测试的主要对照；本项目在Asterinas中重新实现相同磁盘契约，不移植Linux内部对象和代码。
Bento	面向Linux内核的安全Rust文件系统开发框架，关注开发效率、故障隔离、用户态调试和在线升级。	说明Rust可以用于构建接近原生性能的文件系统；本项目进一步面向现有EXT4盘面、原生JBD2和Linux双向互操作。
SquirrelFS	使用Rust typestate检查持久化内存文件系统中的元数据更新顺序和崩溃一致性约束。	其类型化持久化顺序具有启发意义；本项目使用EsInvalidated、RevokeDuty等凭证约束块设备JBD2与Extent修改路径。

续下页

续表1-4

研究或工程对象	主要特点	本项目的借鉴或区别
Theseus	使用Rust和语言内设计方法表达操作系统组件边界、状态依赖和部分系统不变量。	为使用类型和所有权组织内核运行时状态提供参考；本项目把这种方法落实到Linux ABI下的EXT4兼容实现。
开源ext4_rs	使用Rust编写的操作系统无关EXT4实现，提供磁盘结构和部分文件、目录及Extent操作。	作为磁盘格式与接口调研参考；决赛实现独立位于 <code>fs_impls/ext4</code> ，运行时不依赖该库，也不采用外部库包装架构。
Asterinas	采用Rust framekernel和OSTD的操作系统，强调小型且可靠的内存安全TCB，并兼容Linux ABI。	作为本项目的运行平台；原生EXT4重新适配其VFS、Vmo、PageCache、bio和virtio-blk，并显式处理framekernel边界。

表1-4 相关研究与工程对象对比

综合来看，现有工作分别提供了成熟EXT4协议、安全Rust文件系统框架、类型化持久化顺序和framekernel运行平台，但没有直接给出一套可在Asterinas中承担标准EXT4盘面、完整JBD2生命周期、PageCache与O_DIRECT一致性以及Linux互操作的原生实现。本项目继承EXT4磁盘格式与协议语义，同时重新组织缓存、锁、事务和块设备路径，并以EsInvalidated、RevokeDuty等类型凭证表达关键修改前提。

相较于初赛阶段主要证明主体功能和性能可行，决赛版本已经进一步完成原地Extent编辑、EsCache与NodeCache、原生revoke与三阶段恢复，并建立逐FLUSH点断电、数据持久化oracle、checksum/accounting、walcheck和Linux互操作组成的验证体系。因此，本项目的研究贡献不仅是“Rust重写EXT4”，而是给出成熟文件系统协议在RustOS/framekernel环境中的原生实现方法、可验证优化边界和系统化证据链。

2 系统总体设计

2.1 设计目标与总体原则

本项目在Asterinas内核中实现可读写的原生EXT4文件系统，目标不是仅解析EXT4磁盘镜像，而是在VFS、页缓存、内存映射、BIO和块设备接口之上建立完整的文件访问、元数据更新与崩溃恢复能力。设计同时面向磁盘格式兼容性、运行时数据一致性和异常重启后的可恢复性；当三者发生取舍时，以不破坏盘上结构和持久化语义为首要约束。

系统采用原生模块化架构。EXT4与RamFS、ProcFS等实现一样位于Asterinas的`fs_impls`层级，VFS适配代码和EXT4内部磁盘语义通过Rust模块边界与可见性隔离，而不是组织为外部核心库与内核包装层。这样既保留了VFS对象、PageCache和BIO接口的原生集成能力，也避免平台接口侵入超级块、Extent和JBD2等盘上语义。

所有可能修改元数据的操作均从`Ext4::begin_op(credits)`进入事务路径。调用者按照操作类型和Extent树深度预估日志额度，获得`OpHandle`后访问并捕获元数据块，通过`dirty_metadata`或`forget`登记事务结果，最终由运行事务聚合多个操作。未配置日志的文件系统仍经过同一入口完成关闭状态和只读状态检查，避免不同写路径形成彼此独立的修改规则。

系统中的缓存按信息性质分层，而不把任意缓存内容都视为磁盘事实。PageCache保存普通文件数据页；ExtentManager及ExtentTree维护逻辑块到物理块的权威映射；EsCache只缓存已经证明的映射状态，不缓存未经证明的空洞；NodeCache缓存Extent树节点字节；JBD2保留已提交但尚未checkpoint的元数据after-image，使读取者在home block尚未更新时仍能观察到最新的已提交状态。缓存未命中可以退回权威路径重新查询，缓存旧命中则必须通过结构修改时的失效协议消除。

数据持久化采用ordered语义。普通文件数据写入home block，元数据after-image写入日志；提交线程在写入commit block之前等待相关数据写回，并保证descriptor和metadata payload先于commit block持久化。只有具有完整提交记录的事务才会在恢复阶段重放，由此建立文件数据、日志记录与可恢复元数据之间的顺序关系。

并发控制以对象职责和稳定锁序为基础。单inode操作以`Inode::inner`保护文件大小、时间戳和映射相关状态，Extent树结构由自身锁保护，journal state作为叶锁最后获取，checkpoint过程单独串行化；提交线程不获取inode锁，避免后台提交与前台文件操作形成反向依赖。涉及多个inode的namespace操作按照稳定顺序加锁，`O_DIRECT`则在直

接设备访问前排空或失效重叠页，防止页缓存与磁盘形成冲突副本。

未知或不支持的磁盘特性不会被静默忽略，超出已实现深度的Extent变换不会冒险修改磁盘，mmap和HTree写侧等平台或算法限制也在接口处保守处理。错误通过Result向上传播；日志提交失败会中止journal并阻止新的写事务，使局部故障不会继续扩散为不可恢复的全局修改。

2.2 原生EXT4总体架构

原生EXT4实现位于Asterinas VFS与block layer之间，内部围绕数据访问、元数据事务和挂载恢复三条主路径组织。Buffered I/O与mmap通过PageCache访问文件数据，再由ExtentManager完成块映射；O_DIRECT不传输页缓存数据，但仍通过ExtentManager获得或建立映射，最后将连续区间组织为IoBatch和BIO。目录、inode、位图及Extent节点等元数据修改由Handle归入Transaction，经group commit写入JBD2日志，再由checkpoint更新home block；挂载时的recovery复用相同日志格式恢复最后一次一致状态。

图2-1给出上述模块及依赖关系。该图只表达跨层调用和三条主路径，事务提交阶段、缓存不变量与恢复扫描细节在第3章分别展开。

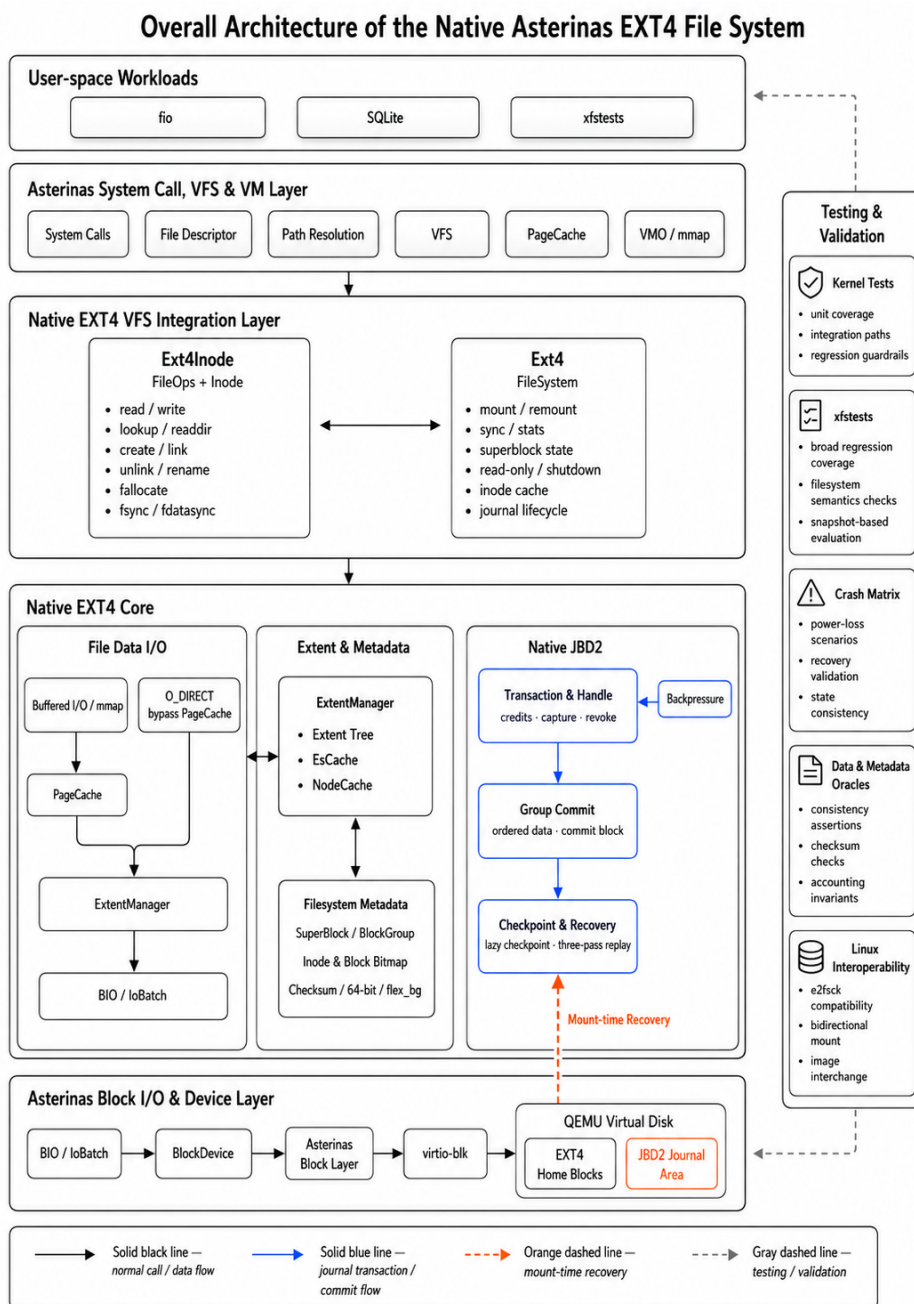


图2-1 Asterinas原生EXT4文件系统总体架构

2.3 代码模块划分

EXT4实现以`kernel/src/fs/fs_impls/ext4/`为根目录，VFS适配、文件语义、块映射、日志和磁盘格式分别由独立模块承担。表2-1列出运行主路径中的代码边界；这种划分强调对象的权威状态归属，也为后续按模块验证锁序、缓存失效与错误传播提供基础。

源码位置（相对ext4/）	核心对象	主要职责
impl_for_vfs/	Ext4Type、Ext4Inode	适配VFS trait，将挂载、文件和目录操作转入EXT4内部对象。
fs.rs	Ext4	管理挂载实例、全局状态、空间分配以及Journal生命周期。
inode/mod.rs	Inode、InodeInner	实现文件I/O、属性维护、fallocate、truncate与同步语义。
inode/dir/	目录对象与HTree读侧	实现lookup、create、unlink、rename等namespace操作及索引目录读取。
inode/extent_manager/mod.rs	ExtentManager、ExtentTree	完成逻辑块映射、Extent查找以及树结构修改。
inode/extent_manager/es.rs	EsCache	缓存已经证明的written、unwritten等映射状态事实。
inode/extent_manager/tree.rs	NodeCache	缓存Extent树节点字节，减少重复的元数据块读取。
journal/	Journal、Transaction、Handle	实现JBD2事务聚合、提交、checkpoint与恢复生命周期。
super_block.rs	SuperBlock	解析和更新超级块、挂载状态与文件系统特性。
block_group.rs	BlockGroup	管理位图、块组描述符、计数器和空间分配。
checksum.rs、feature.rs	校验和与特性位	处理metadata_csum、64bit、flex_bg等格式能力。
kernel/comps/block/（EXT4目录外）	Bio、IoBatch、BlockDevice	承接文件系统块请求，执行批量设备I/O和flush。

表2-1 原生EXT4代码模块划分

2.4 数据路径

运行时数据访问按是否传输PageCache中的数据分为缓存路径和直接路径，但两者共享ExtentManager所维护的块映射。Buffered read在页命中时直接复制数据，未命中时依据Extent映射填充页面；Buffered write先准备可能缺失的映射，再修改缓存页并标脏。mmap通过Vmo与PageCache共享页面，因此其基本读写最终仍进入缓存路径。脏页由写回或同步操作组成连续设备请求，并在需要时与元数据事务建立ordered-data依赖。

O_DIRECT要求偏移和长度按文件系统块对齐。直接读在读取设备前写回重叠脏页，保证此前的Buffered写可见；直接写在inode写锁保护下对已有文件范围调用invalidate_range，该操作先写回脏页再驱逐缓存副本，随后才建立映射并提交直接I/O。空洞写入先分配unwritten extent，数据I/O成功后再将其转换为written，避免未初始化磁盘内容因中途失败而暴露。

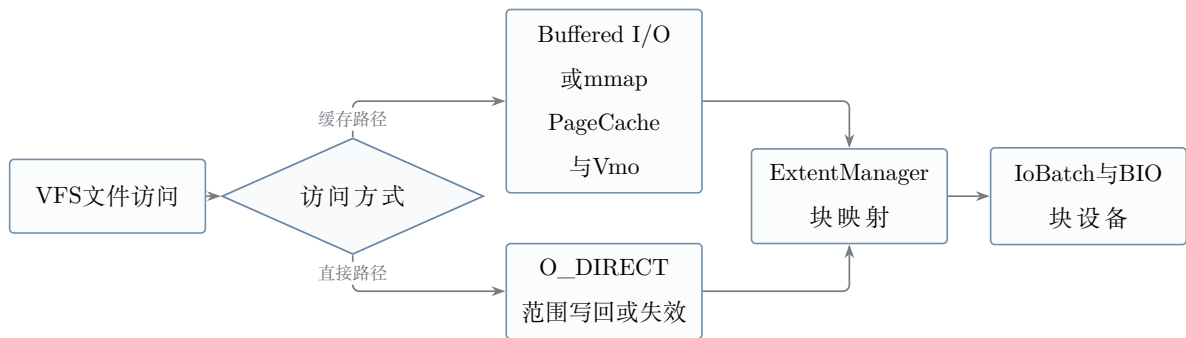


图2-2 文件数据访问的两条主路径

表2-2概括各入口的核心约束。fsync和fdatasync负责等待脏页写回、相关事务提交以及设备刷新完成，为应用提供明确的持久化边界。

入口	主要路径	一致性约束
Buffered read	PageCache命中或依据Extent映射回填页面。	空洞和unwritten extent返回零；映射变化后不得继续使用旧的语义缓存。
Buffered write	准备映射后修改PageCache并标脏，由写回路径提交BIO。	新分配数据在元数据commit之前完成ordered写回。
mmap	Vmo与PageCache共享文件页，缺页及脏页处理复用缓存后端。	当前受反向映射和在途页排空能力限制，结构变换采用保守边界。

续表2-2

入口	主要路径	一致性约束
O_DIRECT read	写回重叠脏页后，根据Extent映射直接读取设备。	偏移和长度按文件系统块对齐，不以旧磁盘内容越过脏缓存页。
O_DIRECT write	排空并失效重叠页，分块完成映射准备、数据I/O和unwritten转换。	inode写锁覆盖一致性窗口，缓存中不保留直接写之前的旧副本。
fsync/fdatasync	写回文件数据，触发并等待目标事务，最后执行必要的设备flush。	返回成功时，调用前要求同步的数据和元数据已经越过相应持久化边界。

表2-2 数据访问入口及一致性约束

2.5 元数据事务路径

创建、删除、重命名、块分配、truncate和Extent转换会同时影响多类元数据，因此不能由各调用点直接、无序地更新home block。操作首先在持有必要对象锁后调用Ext4::begin_op申请日志额度；Handle读取或捕获元数据块的当前版本，修改后登记after-image或forget语义。多个Handle可以聚合到同一个running transaction，由后台提交线程执行group commit，从而在不扩大前台临界区的前提下分摊日志屏障成本。

事务提交成功后，checkpoint再异步更新元数据原始位置；提交与checkpoint之间的版本可见性、日志空间回收和失败降级由JBD2统一处理。本章仅给出这条总体路径，Handle、credits、group commit、revoke和checkpoint的具体状态转换见第3章第3.4节。

2.6 挂载与恢复路径

挂载过程先解析EXT4超级块、块组描述符和journal inode，建立日志几何信息。若超级块表明文件系统需要恢复，则按照JBD2协议依次执行PASS_SCAN、PASS_REVOKE和PASS_REPLAY：第一遍识别具有完整commit record的事务边界，第二遍收集revoke记录，第三遍按事务顺序重放未被撤销的元数据块。三遍分离可以避免扫描尚未完成时就修改home block，也能阻止已经释放或复用的旧块镜像被错误重放。

日志重放可能改变此前已经解析到内存的超级块、块组描述符和位图，因此恢复完

成后必须重新装载这些元数据，不能继续使用恢复前的缓存副本。随后系统完成journal特性与恢复标志处理，启动后台提交线程并发布可用journal，最后执行orphan cleanup，收敛崩溃前未完成的truncate或unlink状态。

若日志几何、校验和、事务序列或重放目标不满足验证条件，挂载流程返回错误，不以猜测方式继续恢复。恢复写入保持幂等，后台提交线程只在恢复和元数据重载完成后启动。三阶段扫描、revoke过滤及运行时启动顺序的实现细节见第3章第3.4节。

3 关键模块设计与实现

3.1 EXT4磁盘结构与特性支持

本项目在Asterinas的kernel/src/fs/fs_impls/ext4/中实现原生EXT4，测试镜像采用4 KiB文件系统块。挂载阶段由SuperBlock解析超级块、特性位和块组描述符；Block-Group负责位图、计数器和块分配；Inode与ExtentManager负责文件数据的逻辑块映射。所有盘上字段在读取边界完成长度、容量和校验和检查，异常结构向上返回错误，而不可信的盘上数据直接转换为内核内存索引。

当前实现覆盖的格式能力如表3-1所示。

能力	实现位置	实现作用
64-bit group descriptor与flex_bg	super_block.rs、block_group.rs	解析描述符高位字段以及可跨块组放置的位图和inode table。
metadata_csum	checksum.rs、各元数据写回入口	以CRC32C校验超级块、块组描述符、位图、inode、目录块和Extent节点。
Extent与unwritten extent	inode/extent_manager/	支持多级树、连续映射及未初始化预分配区间的零值读取语义。
JBD2 checksum v2/v3与revoke	journal/format.rs、journal/revoke.rs	校验日志记录，并阻止已释放块被旧事务中的after-image覆盖。
orphan chain	fs.rs、inode/mod.rs	记录尚未完成的删除和截断；恢复完成后继续回收或重截断。

表3-1 EXT4盘上格式与特性支持

Extent树由根节点、内部索引节点和叶节点构成。Header记录条目数、容量和深度；Index定位下一层节点；叶项以逻辑起始块、物理起始块、长度及written/unwritten状态描述连续区间。相较直接块和多级间接块，Extent可显著减少顺序大文件的映射元数据，也为批量I/O和预分配提供连续物理范围。

超级块解析不是简单地把磁盘字节复制为Rust结构体，而是系统的第一道可信边界。解析过程先检查magic、revision、块大小和inode大小，再处理特性位；随后计算有效的group descriptor宽度、拼接64位块计数、验证块组数量及每组容量，最后在启用meta-data_csum时校验超级块。当前实现要求inode槽能够容纳完整的256字节RawInode，避免按较小槽宽读取时越过当前inode并覆盖相邻描述符。

特性位按EXT4的compat、incompat和ro_compat三类语义处理。compat特性可以在不识别时保持原有数据；未知incompat特性会改变磁盘解释方式，必须拒绝挂载；未知ro_compat特性虽然允许只读解释，但当前文件系统目标是可写挂载，因此返回EROFS，不能在不了解更新规则时修改镜像。代码3-1摘自当前super_block.rs，展示了实际挂载门。

```

1  let unsupported_incompat =
2      sb.feature_incompat & !INCOMPAT_SUPP.bits();
3  if unsupported_incompat != 0 {
4      return_errno_with_message!(
5          Errno::EINVAL,
6          "unsupported incompatible feature"
7      );
8  }
9  let feature_incompat =
10     FeatureIncompatSet::from_bits_truncate(
11         sb.feature_incompat
12     );
13  if !feature_incompat.contains(FeatureIncompatSet::EXTENTS) {
14      return_errno_with_message!(
15          Errno::EINVAL,
16          "ext4 image without the extents feature"
17      );
18  }
19  if sb.feature_ro_compat & !RO_COMPAT_SUPP.bits() != 0 {
20      return_errno_with_message!(
21          Errno::EROFS,
22          "unsupported read-only-compatible feature on a writable mount"
23      );
24  }

```

代码3-1 EXT4挂载时的不兼容特性检查

超级块通过后，块组描述符解析负责把inode位图、块位图和inode table的高低位地址组合为实际物理块号，并验证地址位于文件系统几何范围内。metadata_csum不是只在挂载时检查一次：位图、块组描述符、inode、目录块和Extent节点在各自写回入口重新计算校验和，保证日志中捕获的after-image与最终home block使用相同格式。表3-2给出从盘面输入到可修改内核对象的检查链。

检查层次	主要检查	失败处理
超级块	magic、revision、块大小、inode 大小、特性位和超级块校验和	拒绝挂载，不构造Ext4实例。
文件系统几何	64位块数、块组数、每组容量、描述符宽度及范围运算	返回EINVAL，避免整数截断或越界索引。
块组元数据	描述符地址、位图校验和、空闲计数与inode table位置	返回EIO；不把异常位图交给分配器。
inode与目录	inode校验和、类型、大小、目录项长度及名称边界	当前操作失败，不继续遍历损坏条目。
Extent节点	header magic、深度、容量、条目顺序、逻辑与物理范围	返回EIO，不产生映射或执行部分编辑。

表3-2 EXT4盘面数据的可信边界

3.2 Extent树与空间管理

ExtentManager在ExtentTree锁保护下完成多级树的查找与原位修改。插入时按需分裂叶节点和内部节点、增高根；截断和打洞时原位删除、收缩或合并相邻项，避免将整棵树扁平化后重建。日志credits按树深度和可能触及的元数据块数计算，使一次操作的预约规模随树高增长，而非随整棵树大小增长。

预分配但尚未写入有效文件数据的磁盘块以unwritten状态记录。读取此类区域时，系统按空洞语义返回零；实际写入则先保证映射存在，再在事务保护下将目标范围转换为written状态。

引入unwritten extent后，文件系统可以提前为顺序写入或 fallocate请求批量分配连续磁盘块，在降低频繁空间分配开销的同时，避免尚未初始化的磁盘内容暴露给用户。其主要处理规则如下：

- 读取unwritten extent时返回全零数据，其外部语义与文件空洞一致。
- 写入unwritten extent时，在数据I/O完成后由事务将目标范围转换为written状态，避免尚未初始化的数据对外可见。
- 对顺序追加写，可以在文件尾部提前预分配一段连续块，减少每写入一个4 KiB数

据块就执行一次空间分配和Extent更新所产生的元数据开销。

- Extent树解析过程中如果发现节点深度、条目数量或映射范围异常，底层函数返回EIO，而不是通过panic终止整个内核运行。

Extent查找首先在根节点中定位不大于目标逻辑块的最后一个Index，逐层下降到叶节点，然后判断目标落在已有Extent、相邻空洞还是树外区域。读操作只消费查找结果；修改操作必须先使EsCache对应范围失效，再在叶节点执行分割、缩短、删除或插入。如果叶节点没有空位，make_room_for沿访问路径向上分裂：新建右兄弟、修正父Index的逻辑起点，必要时提升根深度。每个被修改或新分配的节点都通过当前Handle捕获，不能绕过journal 直接写回。

操作	树内处理	空间与日志副作用	完成条件
查询映射	按Index逐层下降并检查叶项覆盖关系	无元数据修改；命中节点可进入NodeCache	返回written、unwritten或hole。
插入映射	合并相邻兼容Extent；空间不足时分裂叶与祖先	分配外部节点时更新位图、块组描述符和超级块	新映射及父Index均已被Handle捕获。
转换written	在边界处分裂unwritten项，将完成I/O的中段改为written	通常只重写现有叶；边界分裂可能增加条目	数据BIO成功后才能执行转换。
truncate或punch	缩短、删除或拆分相交Extent，并修正祖先键	释放块前登记必要revoke，块在事务提交前保持pinned	映射撤销、计数器和inode大小处于同一可恢复顺序。
collapse或insert	对后续逻辑块整体左移或右移，重新序列化受影响树形	操作前一次性检查深度和credit是否可容纳	任一前置条件不满足时不修改树。

表3-3 Extent操作的原地编辑步骤

文件逻辑块在数据路径中呈现hole、unwritten和written三种状态。图3-1中的关键约束是：分配首先产生unwritten映射，只有数据BIO完成后才允许转换为written；释放则先撤销映射并记录必要的revoke，再使物理块重新进入分配器。该顺序使崩溃后的可

见状态只能是零值区间或完整数据，不会把未初始化块解释为有效文件内容。

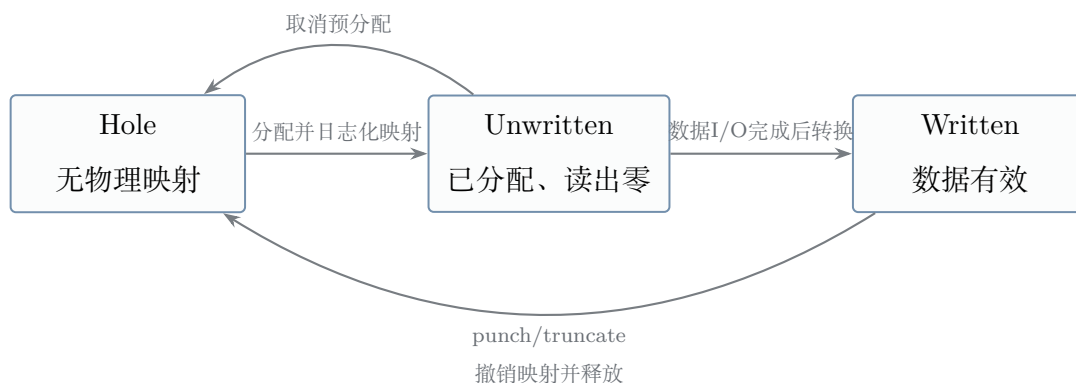


图3-1 Extent三态及其转换约束

实现不会为所有Extent插入操作预留相同数量的日志credit，而是根据当前树深和可能发生的逐层分裂计算所需额度。一次原地插入最多需要改写访问路径上的旧节点、为各层分裂分配新节点并更新祖先键；只有新分配的节点和数据Extent会额外修改块位图与块组描述符。实现将位图项限制在块组总数内，将描述符项限制在GDT块总数内，得到与文件Extent总量无关、随树深增长的上界。代码3-2摘自fs.rs中的当前计算。

```

1 pub(super) fn insert_credit_bound(
2     &self,
3     depth: u16,
4 ) -> usize {
5     let grown = depth as usize + 1;
6     let node_writes = 3 * grown + 2;
7     let allocs = grown + 2;
8     let bitmaps = allocs.min(self.nr_groups());
9     let gdt = allocs.min(self.nr_gdt_blocks());
10    node_writes + bitmaps + gdt
11        + Self::SUPERBLOCK_CREDITS
12 }
13
14 pub(super) fn chunk_insert_credits(
15     &self,
16     depth: u16,
17 ) -> usize {
18     self.insert_credit_bound(depth)
19         + Self::INODE_DESC_CREDITS
20 }

```

代码3-2 原地Extent插入的日志credit上界

连续写入、预分配和大范围截断可能超过单个事务的最大credit。此时实现按有界chunk推进：每个chunk在进入树锁前确认剩余额度，完成映射、数据I/O与必要转换后结束当前事务；下一 chunk在释放ExtentTree锁后通过journal_restart进入新事务。若单次插入的理论上界已经大于journal允许的max_credits，操作在分配任何块之前失败，避免形成无法提交的半成品树。

在VFS提供的fallocate接口上，当前实现覆盖五类操作，具体语义与边界如表3-4所示。

操作	Extent处理	文件大小与边界
Preallocate	为空洞分配连续物理块并插入unwritten extent。	普通模式可扩展文件；KEEP_SIZE不改变i_size。
Punch hole	处理非对齐边缘后，删除完整块区间的映射并释放空间。	要求KEEP_SIZE，文件大小不变。
Zero range	边缘块原位写零，完整块区间转换或建立为unwritten。	支持普通模式和KEEP_SIZE。
Collapse range	删除区间并将后续逻辑映射整体左移。	要求块对齐；深度超过2的树当前受限。
Insert range	将后续逻辑映射右移，在目标位置打开空洞。	要求块对齐；深度超过2的树当前受限。

表3-4 fallocate操作的Extent处理

常规Extent操作采用多级树原位调整；collapse/insert的深树限制是为保持单事务原子性而设置的显式边界。上述机制为buffered I/O、O_DIRECT和空间回收提供统一映射基础。

3.3 EsCache与NodeCache

原生EXT4在Extent映射路径中使用两类职责不同的缓存。EsCache (Extent Status Cache) 记录经过Extent树查询或修改后确认的逻辑块映射状态，用于快速判断目标范围是否全部为written或至少已经完成映射。缓存未命中只会退回Extent树查询，而不会被解释为文件空洞；所有可能改变逻辑映射含义的树修改，都必须先对相应范围执行失效。

EsCache的查询结果分为AllWritten、AllMapped和Unknown：前两者分别证明范围内全为已写入映射、或全部已有映射但包含unwritten；Unknown 只表示缺少已证明事实，必须回退到树遍历。缓存从不记录“这是空洞”这一事实，从源头避免旧缓存将有效数据误读为零。每个逻辑修改在编辑叶项前取得 EsInvalidated类型凭证；调试构建对非Unknown结果重新遍历权威树校验。

NodeCache缓存外部Extent节点的块内容，减少热点文件反复访问Extent叶节点和索引节点时产生的设备读取。节点写回后同步刷新缓存，节点被释放或复用时清理相应缓存状态。每个inode维护8个节点槽位，命中时直接复用已验证的4 KiB节点字节；调试构建下，命中会与新的设备读取交叉核验。两类缓存均属于ExtentTree锁保护的内容：EsCache是语义事实缓存，NodeCache是节点字节缓存，二者不能互相替代；其内部短临界区不会跨设备I/O或日志操作持有。

两类缓存的正确性边界可归纳为表3-5。设计上的共同原则是“允许漏报、不允许误报”：缓存清空或过度失效只会带来一次慢路径查询，错误命中则可能绕过块分配或unwritten转换，因此必须由锁、失效协议和调试复核共同约束。

比较项	EsCache	NodeCache
缓存内容	逻辑块到物理块及written/unwritten状态的已证明事实。	外部Extent节点的完整块字节。
未命中处理	返回Unknown并重新遍历Extent树。	经JBD2读取漏斗或设备读取节点并填充槽位。
更新协议	逻辑修改前失效范围，修改后只记录刚被证明的事实。	节点写回时刷新；节点释放、复用或全树重建时清空。
验证机制	调试构建重新遍历树，复核AllWritten/AllMapped结论。	调试构建以新鲜读取结果逐字节核对命中内容。

表3-5 EsCache与NodeCache的分层职责

3.4 JBD2日志子系统

JBD2日志子系统是本项目实现强一致性语义的核心模块。原生实现以Journal、Transaction和Handle组织元数据修改。文件系统操作通过Ext4::begin_op(credits)预约日志空间并取得OpHandle；随后捕获将被修改的元数据after-image，标记dirty或forget，最后由运行事务聚合提交。提交完成前home block不承担提交点职责，已提交的after-image由checkpoint异步写回其原始位置。在commit和checkpoint之间，设备上的home block仍可能落后于最新提交事务；因此元数据再次被捕获时，读取漏斗优先使用UncheckpointedImage，其次才读取设备。该保留层避免后续事务以旧home block 为

基础生成新的after-image，是lazy checkpoint能够安全存在的必要条件。

图3-2给出核心对象关系。credits是对本次操作可能修改的元数据块数的保守预约：预约不足或不可恢复的元数据错误会终止日志并使文件系统拒绝后续写操作，避免以静默丢失元数据的方式继续运行。

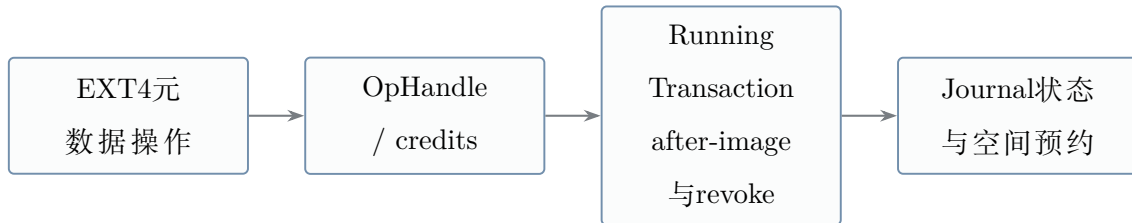


图3-2 JBD2事务对象关系

JBD2事务从运行、提交、检查点到恢复的主要阶段如表3-6所示。

阶段	作用	实现要点
Running	收集当前事务涉及的元数据块镜像。	Handle在credits范围内捕获after-image；ordered数据写入被登记，对应home block写回被推迟到事务提交之后。
Commit	将descriptor、metadata payload和commit block按规定格式写入journal。	group commit汇集运行事务；descriptor可形成链，revoke记录同批写入；释放块在释放事务提交前保持pinned，barrier后写入带校验和的commit block。
Checkpoint	将已经提交事务中的元数据镜像写回对应的home block。	lazy checkpoint按事务顺序写回home block、淘汰相应的保留after-image并释放日志空间。
Recovery	文件系统挂载时扫描journal，并重放已经完整提交的事务。	依次扫描、收集revoke、重放完整已提交事务；之后重新装载元数据并清理orphan链。

表3-6 JBD2事务处理阶段

JBD2环形日志由journal超级块、descriptor block、元数据payload、revoke block和

commit block组成。descriptor通过tag给出后续payload对应的文件系统块号；payload保存提交时刻的完整after-image；revoke记录表示某块的旧日志镜像不得再重放；commit block 携带事务序号和校验和，是恢复侧判断事务完整性的唯一提交点。仅看到descriptor或payload 不能推断事务已经提交。

日志记录	写入内容与时机	恢复语义
Descriptor	transaction序号及一个或多个目标home block tag	指定payload归属；本身不表示事务提交。
Metadata payload	inode、位图、块组描述符、目录块或Extent节点的after-image	仅在同事务存在有效commit block时进入重放候选。
Revoke	被释放或复用的元数据块号及事务序号	PASS_REVOKE收集后，抑制较旧事务对该块的重放。
Commit block	transaction序号、提交时间及事务校验和	校验通过后建立已提交边界；不完整尾事务被忽略。
Journal superblock	环形日志起点、序号和特性信息	定位扫描窗口；checkpoint推进后更新可回收起点。

表3-7 JBD2日志记录及其恢复语义

revoke写侧与恢复侧使用同一事务序号协议。运行事务先在私有集合中登记需撤销的块；提交线程把集合编码为一个或多个revoke block，但此时仍不能向恢复表发布；只有commit block通过第二次barrier持久化后，该集合才成为已提交RevokeTable的一部分。元数据块释放要求调用者携带RevokeDuty，并由BlockFreeAuth证明释放策略；普通文件数据不被JBD2作为元数据after-image重放，因此可以使用无revoke义务的授权。类型区分使新增释放路径必须显式回答“旧日志是否可能覆盖该块”。

日志空间管理同时跟踪已使用环形块、运行事务的预约credit和已提交但未checkpoint的事务。前台Handle不能把“物理上尚有空块”等同于“日志上可以安全覆盖”：只有checkpoint释放的区间才可复用。空间不足时，前台请求提交running事务，并在必要时推进checkpoint；等待前必须释放inode与ExtentTree等上层锁。若提交线程检测到记录无法装入日志、设备I/O失败或校验和构造失败，则abort journal，唤醒等待者并使后续begin_op返回EROFS。

提交线程不持有inode锁。它先等待ordered模式下登记的数据I/O完成，再连续写入

descriptor、元数据after-image和revoke记录；完成barrier后写入commit block，以该记录作为事务可恢复的边界。checkpoint与提交解耦：前者可延迟并批量执行，后者在日志空间不足时施加backpressure，从而在吞吐、内存占用和可恢复性之间保持平衡，如图3-3所示。

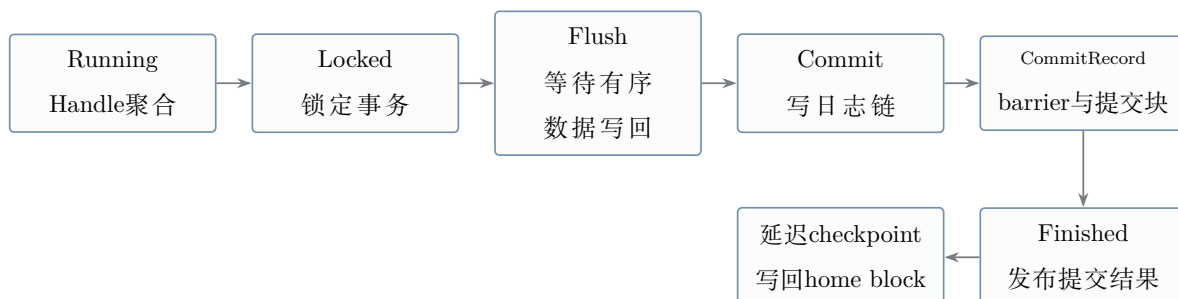


图3-3 JBD2提交与checkpoint流程

提交阶段的合法后继关系在代码中集中定义，任何跳跃或逆序转换都会被上层状态检查拒绝。代码3-3来自当前 `journal/transaction.rs`，与图3-3中的状态顺序一致。

```

1  impl CommitPhase {
2      pub(super) fn next_in_pipeline(self) -> Option<Self> {
3          match self {
4              Self::Locked => Some(Self::Flush),
5              Self::Flush => Some(Self::Commit),
6              Self::Commit => Some(Self::CommitRecord),
7              Self::CommitRecord => Some(Self::Finished),
8              Self::Finished => None,
9          }
10     }
11 }
  
```

代码3-3 JBD2提交阶段的合法后继关系

挂载恢复严格执行PASS_SCAN、PASS_REVOKE和PASS_REPLAY：先识别完整提交事务，再收集其revoke集合，最后仅重放未被revoke覆盖的after-image。恢复完成后重新读取超级块和块组元数据，启动提交线程并清理orphan链；缺少有效commit block的不完整事务不会进入重放路径。日志校验和不匹配或不可恢复错误触发abort/只读降级，而非继续写入。

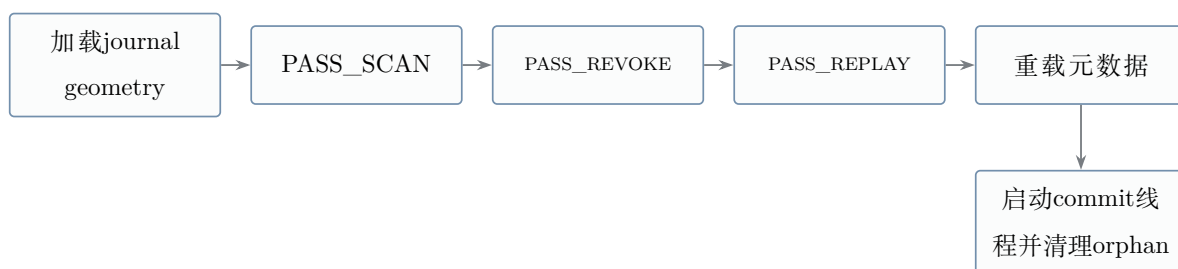


图3-4 JBD2挂载恢复流程

标准JBD2按照主机可能随时失去供电的条件设计，因此每个提交事务都要把descriptor、元数据after-image和commit block写入日志，并使用设备屏障建立可恢复的持久化顺序。这套机制适用于缺少供电保障的通用环境，但在小块写入并频繁调用fsync时，重复写入日志元数据和等待屏障会带来明显开销。

供电保护已经广泛进入现代计算设备。数据中心通常采用UPS、备用发电机或机架级电池备份单元维持供电；带有电池的终端设备也具备完成受控关机的条件。Google公布的资料显示，其全球数据中心机架电池组已经部署超过一亿颗锂电芯，这些电池既用于跨越短时电力扰动，也能在长时间失电时为安全关机提供时间[15]。Open Compute Project的Open Rack V3规范进一步给出了由6个电池备份模块组成的5+1冗余方案，明确用于交流电源中断期间继续为系统负载供电[16]。微软公开的数据中心架构同样将UPS与应急发电机作为连续运行的供电保障[17]。本项目据此假设部署侧能够在外部供电中断后发出通知，并提供约3-5分钟的备用供电窗口，使文件系统有时间完成一致性收敛和安全关机。

外部电源保护模式并未忽略掉电，而是把持久化工作从每次事务提交转移到掉电通知后的有限供电窗口。正常运行时，文件数据仍按照ordered语义完成写回和设备屏障；事务状态机也继续运行，只有达到Finished状态的事务才被视为已经完成。与标准模式不同的是，journal descriptor、元数据payload和commit block不在每次提交时写入日志区，完成事务的元数据新版本保存在uncheckpointed映射中，运行期间不执行metadata checkpoint。

收到UPS掉电通知后，系统不能把所有内存脏数据直接写回磁盘。此时某个文件操作可能只完成了Extent分裂、inode更新或块位图修改的一部分，直接写回会把半完成状态变成永久的盘面结构。当前实现通过EXT4_IOC_SHUTDOWN进入掉电处理路径：首先设置文件系统shutdown标志，使后续begin_op返回EIO；然后停止并等待提交线程退出，保证内存事务集合不再变化；最后按照事务阶段决定保留或丢弃哪些元数据镜像。

Running和Locking事务尚未完成提交，其私有after-image直接丢弃。已经到达Finished状态的事务满足ordered data先行和元数据集合完整两个条件，对应镜像可以写回

home block。若Committing事务停在Finished之前，或者Journal此前已经abort，系统无法证明uncheckpointed映射中的合并结果仍代表完整事务序列，此时会清空保留镜像、revoke表和延迟释放记录，回退到最近一次磁盘一致状态。这里的“回滚”不是在磁盘上执行逆向修改，而是丢弃从未写入home block的不完整内存版本；只有确认完整的事务才进入批量写回，最后执行flush和barrier并停止文件系统服务。

图3-5给出掉电通知后的处理流程。需要写回的数据是Finished事务已经按照ordered顺序完成的数据，不包括无法确认完整性的任意脏页或Active事务数据。

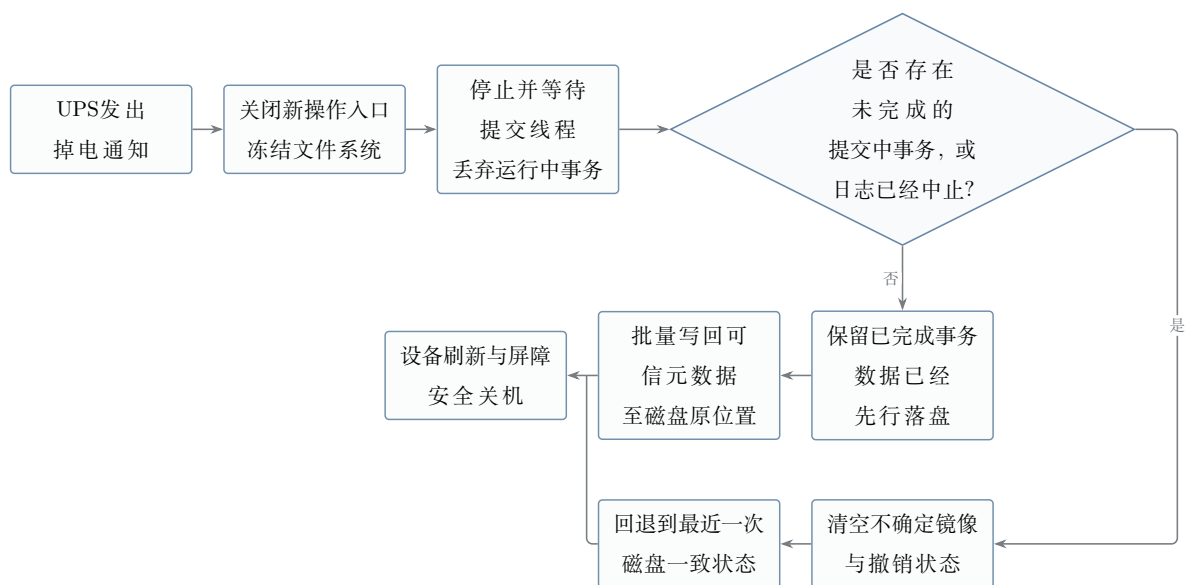


图3-5 外部电源保护模式的掉电通知与事务收敛流程

代码3-4摘自journal/mod.rs。代码先删除未完成事务的内存席位，再检查Committing事务是否达到Finished；无法证明完整性时，宁可回到最近的磁盘一致状态，也不写回不确定的元数据镜像。

```

1 let discard_all_retained = {
2     let mut state = self.state_write();
3     state.running = None;
4     state.locking = None;
5
6     match state.committing.take() {
7         None => false,
8         Some(committing) if committing.phase() == CommitPhase::Finished => false
9     },
10    Some(_) => {
11        state.uncheckpointed.clear();
12        state.revoked = revoke::RevokeTable::new();
13        state.pinned_frees.clear();
  
```

```
13         true
14     }
15 }
16 };
17
18 if !discard_all_retained {
19     self.persist_volatile_metadata_on_unmount()?;
20 }
21 self.abort();
```

代码3-4 掉电通知后的不完整事务回滚与可信元数据写回

比较项	标准JBD2模式	外部电源保护模式
正常运行时的数据	ordered data写回并执行持久化屏障	保持相同的数据写回和屏障，Finished事务的数据已经先行落盘
正常运行时的日志与元数据	写入descriptor、元数据payload和commit block，随后执行checkpoint	完成事务的元数据镜像保存在内存，不写日志区和home block
掉电通知后的入口控制	按标准shutdown或日志恢复路径处理	关闭新操作入口，停止并等待提交线程，固定内存事务集合
未完成事务	无完整commit block的事务在恢复时不重放	丢弃Running和Locking事务；未完成Committing事务使系统回到最近一次磁盘一致状态
完成事务	通过日志重放或checkpoint写回	仅把Finished事务对应的可信元数据批量写回home block
完成条件	commit block和设备持久化顺序成立	写回可信元数据后执行flush和barrier，并在备用供电窗口内安全关机

表3-8 标准JBD2与外部电源保护模式的提交差异

该模式默认关闭，通过内核命令行参数ext4.power_protected=1启用，构建运行命令为make run_kernel EXT4_POWER_PROTECTED=1。UPS通知通过EXT4_IOC_S

HUTDOWN进入文件系统关闭路径。启用后，fsync和fdatsync仍等待文件数据写回及事务状态推进，但元数据持久化完成点移动到掉电通知或受控卸载阶段。该方案依赖通知能够送达、备用供电窗口足够完成回滚和写回，并且内存与内核仍可可靠执行；无法满足这些条件时，应继续使用默认的标准JBD2模式。

3.5 Buffered I/O与PageCache

真实应用通常通过普通read/write、mmap和fsync完成文件访问与持久化，而不是完全围绕O_DIRECT组织I/O。为支持这类真实负载，本项目将Asterinas通用PageCache接入EXT4。普通buffered I/O经由页缓存、ExtentManager和BIO访问设备；fsync/fdatasync负责推进脏页写回并等待相关事务；O_DIRECT绕过页缓存传输数据，但必须在重叠范围内与缓存协调。mmap通过Vmo接入同一基础设施，其复杂组合场景的边界在第3.7节单独说明。

普通buffered read/write通过VFS适配层访问文件页。读路径对空洞和unwritten extent返回零；写路径先取得或建立逻辑块映射，再更新页缓存并标记dirty。对连续页范围，写回阶段合并为批量BIO，减少逐页映射查询与小I/O提交的开销。

读路径维护按inode划分的顺序访问状态。首次或随机读取只预取请求本身；识别到连续访问后，系统在剩余预取窗口低于阈值时扩展前沿，并将窗口逐步增大到上限。PageCache::prefetch_range负责异步填充，底层读取将物理连续的Extent段加入IoBatch后统一等待。O_DIRECT不更新该顺序检测器，避免绕缓存访问污染buffered read的预读状态。

对于已经具有稳定Extent映射的文件范围，覆盖写可以直接更新对应缓存页，并将页面标记为dirty。如果写入涉及文件追加、空洞填充或新块分配，则需要先完成必要的Extent映射和inode size更新，再将数据写入PageCache。其中，块分配、Extent更新和inode更新等元数据修改，通过相应的日志化准备过程完成。该路径的主要处理流程如图3-6所示。

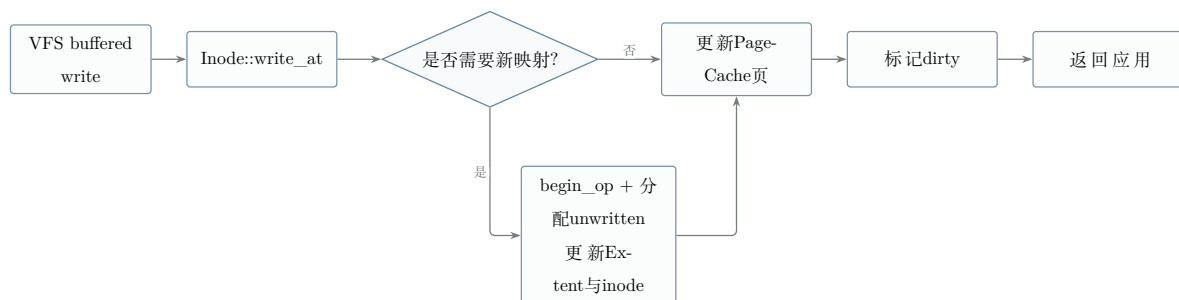


图3-6 Buffered write路径

普通write完成后，数据已经进入PageCache，但页面仍处于dirty状态，因此不能认为数据已经完成持久化。后续需要通过fsync或同步写回流程，将缓存中的修改推进到块设备。

fsync/fdatasync是buffered I/O的持久化边界。执行fsync时，系统首先收集目标文件中的脏页，并将逻辑上连续的脏页组织为连续区间。这些区间随后以批量方式写回，从而减少逐页进行Extent映射查询以及提交大量4 KiB bio所产生的软件开销。

文件数据写回完成后，系统提交并等待与该文件相关的必要JBD2事务tid，随后执行设备flush。当上述过程完成后，已经写回的页面由dirty状态转为clean状态，但页面本身继续保留在PageCache中。fsync的主要处理流程如图3-7所示。

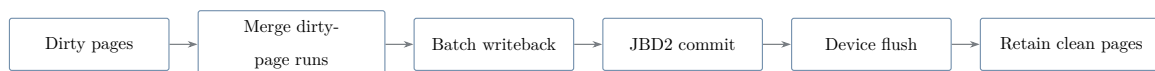


图3-7 fsync持久化路径

写回和事务等待是两个不同阶段。PageCache把dirty页写到其home data block，只能证明数据 I/O已经完成；Extent、i_size和inode描述符仍可能只存在于尚未提交的JBD2事务中。因此每个inode维护sync_tid和datasync_tid：前者记录fsync必须等待的最新元数据事务，后者只记录读取文件数据所必需的事务。纯chmod、chown或utimens只推进 sync_tid；数据、映射或文件大小变化同时推进两者。

新建文件是容易遗漏的边界。创建事务包含目录项、新inode描述符和空Extent根，即使随后没有再次修改inode，fdatasync也必须等待创建事务，否则应用确认成功的文件可能在崩溃后消失。当前实现因此在inode发布前同时记录两个tid。代码3-5来自 inode/mod.rs。

```

1 pub(super) fn record_create_tid(
2     &self,
3     handle: Option<&journal::Handle>,
4 ) {
5     if let Some(handle) = handle {
6         let mut inner = self.inner.write();
7         inner.sync_tid = Some(handle.tid());
8         inner.datasync_tid = Some(handle.tid());
9     }
10 }
  
```

代码3-5 新建inode同时记录fsync与fdatasync目标

inode变化	sync_tid	datasync_tid	等待原因
创建inode与目录项	推进	推进	文件存在性是其数据可读的前提。
写入、truncate或Extent变化	推进	推进	数据位置、有效长度或文件大小发生变化。
纯属性变化	推进	不推进	fsync覆盖完整元数据，fdatasync无需强制无关属性。
无脏页且无新捕获	保留既有值	保留既有值	仍需等待先前捕获但尚未提交的事务。

表3-9 fsync与fdatasync事务号更新规则

调用sync_data_and_meta时，系统在inode锁内完成脏页写回和必要的inode after-image捕获，记录需要等待的tid，然后释放所有文件系统对象锁。随后log_wait_commit等待目标事务，最后执行块设备sync。提交线程不获取inode锁，等待方也不带锁睡眠，因此前台fsync和后台commit之间不会形成反向锁依赖。

早期实现曾在fsync完成后清空整个文件缓存。该策略虽然移除了已经写回的脏页，但同时也会丢弃仍然有效的clean页，使SQLite等频繁执行提交操作的应用不断重新从设备读取工作集。

因此，当前实现不再在每次fsync后执行整体缓存清空，而是仅清除页面的dirty状态，并只在确有一致性要求时失效指定范围内的缓存页。这样既能够完成持久化，又可以保留后续访问仍然需要的缓存工作集。

3.6 O_DIRECT与缓存一致性

与buffered I/O不同，O_DIRECT请求根据Extent映射直接组织bio，并将数据提交到块设备，不通过PageCache传输。但是，绕过PageCache并不意味着direct I/O可以忽略缓存中已经存在的数据状态。

所有O_DIRECT请求首先检查文件偏移和传输长度的文件系统块对齐条件，不满足时返回EINVAL。对于direct read，如果待读取范围内仍存在尚未写回的脏页，直接读取块设备可能得到旧数据。因此direct read先对重叠范围执行flush_range，再读取设备。对于direct write，如果重叠范围仍保留脏页，这些页的后续写回可能覆盖direct write结

果；如果保留clean页，后续buffered read又可能读到旧副本。当前实现在inode写锁内、提交数据BIO前调用invalidate_range，该操作先完成重叠脏页写回，再逐出重叠缓存页。写锁一直覆盖映射、数据I/O和元数据更新，因而在direct write完成前不会有并发buffered writer 重新弄脏该范围。

direct write遇到空洞时先分配unwritten extent；每个有界chunk将物理连续段加入IoBatch，等待全部数据BIO完成后，再在同一事务内把相应区间转换为written并捕获inode描述符。事务提交前的设备barrier保证数据先于引用它的元数据提交，形成O_DIRECT下的ordered-data约束。代码3-6摘自当前inode/mod.rs中的write_direct_at_once：inode写锁在缓存失效前取得，随后才开启具有深度相关credits的事务。

```

1 let mut inner = self.inner.write();
2 let old_size = inner.file_size();
3
4 let discard_end = end.min(old_size);
5 if offset < discard_end {
6     inner
7         .page_cache()?
8         .invalidate_range(offset..discard_end)?;
9 }
10
11 let depth = inner
12     .extent_manager()
13     .map(|em| em.root_depth())
14     .unwrap_or(0);
15 let mut op = fs.begin_op(fs.write_credits(depth))?;
16 self.write_direct_chunked(
17     fs, offset, end, reader, &mut op, &mut inner,
18 )

```

代码3-6 O_DIRECT写入前的缓存协调与事务入口

该一致性处理路径如图3-8所示。

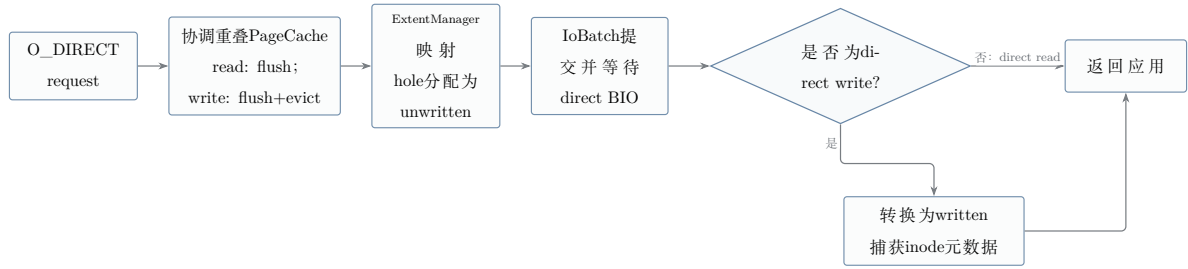


图3-8 O_DIRECT请求与页缓存一致性处理路径

综上，本项目围绕PageCache建立buffered I/O的访问与持久化语义：fsync负责将脏页及相关事务状态持久化；O_DIRECT则在绕过缓存传输数据的同时，通过访问前的范围写回/失效协议维护页缓存与块设备之间的一致性。相关机制总结如表3-10所示。

机制	作用	实现说明
Buffered I/O页缓存	统一普通读写的数据页、脏页跟踪和写回。	空洞和unwritten范围按零值读取；连续脏页合并后经ExtentManager组织批量BIO。
连续脏页批量写回	减少逐页映射查询和大量小bio带来的软件开销。	fsync将逻辑上连续的dirty page合并为写回区间，再进行批量提交。
fsync保留clean页	避免频繁提交场景反复重建缓存工作集。	写回完成后仅清除dirty状态，不再使用早期的evict_all清空整个缓存。
O_DIRECT对齐与映射	保证绕缓存I/O按块设备语义执行。	检查偏移和长度对齐；空洞先分配，unwritten范围在数据完成后转换。
O_DIRECT一致性协议	防止direct I/O与PageCache之间出现新旧数据不一致。	direct read前写回重叠脏页；direct write在inode写锁内先写回并逐出重叠页，再提交数据BIO。

表3-10 PageCache与O_DIRECT关键机制

3.7 mmap基础路径与一致性边界

mmap通过Asterinas的Vmo与PageCache接入EXT4，基础映射读取和普通脏页写回可以复用buffered I/O的数据路径。VFS取得inode对应的PageCache后，将同一缓存对象暴露给文件映射；缺页读取因而与普通 buffered read使用相同Extent映射，MAP_SHARED产生的脏页也进入同一写回集合。这样可以保证普通read/write和基础mmap访问不会各自维护互不相干的数据副本。

mmap一致性的困难主要出现在文件映射发生结构变化时。文件系统能够从inode找到PageCache，却需要VM层进一步回答：某个文件页当前映射到哪些用户页表、是否存在正在执行的缺页或写回、何时可以安全撤销页表映射。Asterinas当前尚未向文件系统提供完整的反向映射和统一的在途页排空协议，因此本项目只对能够建立缓存可见性证明的组合开放操作。

组合场景	当前处理	需要维持的不变量
mmap读取与 buffered read	共享同一PageCache和Extent查询路径	相同文件偏移观察同一缓存页内容。
MAP_SHARED写与fsync	脏页进入普通写回集合，由fsync推进数据和事务	fsync返回前数据块、必要元数据和设备barrier完成。
文件扩展	先建立映射并更新大小，再允许新范围被缓存访问	不把未初始化物理块暴露为文件数据。
truncate或打洞	对能够安全排空的缓存范围执行写回/失效；无法证明时保守拒绝组合	被删除范围不能继续由旧页表映射访问。
collapse/insert range	受反向映射与在途页排空限制，不对活动映射执行未经证明的页偏移重排	逻辑偏移变化后不得保留指向旧数据的用户映射。
与O_DIRECT重叠写	仅在重叠缓存范围能够排空并失效时进入直接写路径	direct write完成后不存在可读到旧内容的缓存或用户映射。

表3-11 mmap组合访问的一致性边界

因此，当前目标范围内的基础文件映射、读取、共享页修改和普通同步路径已经完

成；对 truncate、区间移动或O_DIRECT重叠等复杂组合，接口采取保守同步或显式拒绝。这里的限制是文件页生命周期协议的边界，而不是EXT4无法解析相应磁盘块。后续需要在Asterinas VM层补充文件页反向映射、页表失效回调和在途操作等待，才能将这些组合提升到与普通PageCache 路径相同的支持等级。

3.8 并发、锁序与错误处理

并发设计以可证明的锁序为优先目标。当前跨层顺序为：inode的inner锁、journal handle、ExtentManager::state、s_orphan_lock、超级块锁和块组元数据锁。journal state是元数据访问漏斗最后取得的叶锁，不跨等待持有；EsCache和NodeCache的内部自旋锁同样只覆盖探测或更新，不跨设备I/O、日志预约和checkpoint。

涉及多个inode的namespace操作按稳定的inode号顺序取得对象锁，避免ABBA死锁。结构性修改（分配、截断、目录项修改、Extent调整）在排他保护下进行；checkpoint串行推进已提交事务，O_DIRECT不依赖早期的“共享锁覆盖写”假设，而是在inode写锁内先写回并逐出重叠页，再执行映射和设备I/O，以此维护与PageCache的一致性。

对象锁解决运行时竞争，Handle解决崩溃后的原子可恢复性，两者不能互相替代。create需要同时修改父目录块、新inode位图、inode table和计数器；rename可能同时修改两个目录及被覆盖目标；truncate还会跨Extent树、位图和orphan链。实现先按稳定次序取得对象锁，再以操作专用credit开启Handle，使并发观察顺序和磁盘提交边界保持一致。

操作	并发保护	事务覆盖对象	异常处理重点
create/link	父目录与目标inode按对象规则加锁	目录块、inode位图、inode table、块组与超级块计数	名称发布和新inode存在性必须由同一事务承载。
unlink/rmdir	父目录与被删除inode保持稳定锁序	目录项、链接计数、orphan头及相关inode描述符	链接数归零后先进入orphan，再分块回收数据。
rename	涉及的目录和inode按inode号排序	旧目录项、新目录项、移动inode及可能被覆盖目标	全部前置检查在修改前完成，避免只移动一侧名称。
truncate/ fallocate	inode写锁与ExtentTree排他修改	Extent节点、位图、GDT、超级块、inode与revoke	大操作按credit分块，orphan记录保证崩溃后可继续清理。
fsync/ O_DIRECT	inode锁内快照缓存和目标tid，等待前释放锁	inode after-image及数据相关事务；直接写另含映射转换	设备失败返回EIO，日志失败abort并拒绝后续写入。

表3-12 典型操作的锁与事务边界

全局锁序及其约束如表3-13所示。

顺序位置	锁或对象	约束
1	inode inner	保护文件大小、属性、PageCache句柄和inode级操作；多inode操作按inode号升序获取。
2	journal handle	在inode锁之后通过begin_op取得；仅预约和捕获，不在持有更内层锁时等待空间。
3	ExtentManager::state	保护ExtentTree、EsCache和NodeCache；需要事务重启时先释放此锁。

续下页

续表3-13

顺序位置	锁或对象	约束
4-6	orphan链、超级块、块组元数据	按此顺序更新跨inode和分配器状态；journal state只作为短时叶锁进入，不跨I/O等待。

表3-13 原生EXT4跨层锁序

文件系统与普通应用不同，它不能假设磁盘上的结构始终合法。目录项可能已经损坏，Extent节点可能越界，块位图可能与inode记录不一致，一个操作也可能在ENOSPC或I/O错误处只完成部分步骤。原生实现将盘上数据视为不可信输入：解析边界验证magic、节点深度、条目容量、块号范围、整数运算和校验和，不能证明合法的结构返回EIO，不以panic处理可预期的介质错误。

空间不足并不简单等同于磁盘无空闲块：块可能正被尚未提交的释放事务pinned。分配路径在这种情况下请求提交相关事务并重试；日志预约在进入修改前核对credits与环形空间，必要时触发commit/checkpoint backpressure。长操作通过有界chunk推进，credits不足时先退出ExtentTree临界区，再重启handle，避免在树锁内等待日志空间。

对于已经发生不可逆内存修改、但相应after-image无法纳入事务的错误，单纯向调用者返回失败仍可能让后续事务建立在不一致状态之上。因此这些元数据漏斗调用abort_journal_on_fs_error：终止当前日志，后续写操作在begin_op统一返回EROFS，读操作仍可用于诊断和数据导出。主动EXT4_IOC_SHUTDOWN则使用独立的shutdown状态，使新的写操作返回EIO；两者分别表示错误降级与管理员强制停止写入。

主要风险及其当前处理方式如表3-14所示。

风险类型	可能后果	当前处理
Extent节点损坏	entries_count越界，导致切片访问越界、错误映射或遍历死循环。	对节点容量和条目数量进行钳制检查，发现异常时返回EIO。
ENOSPC中途失败	日志空间或已释放但仍pinned的块暂时不可用，长操作可能只完成前缀。	在修改前预约credits；空间压力触发commit/checkpoint，长操作按chunk重启事务。

续下页

续表3-14

风险类型	可能后果	当前处理
缓存旧映射	将文件空洞或unwritten区间错误判断为written。	保持coverage $\subseteq$ truth，无法确认映射有效时退回慢路径重新查询。
不可逆修改后的日志失败	内存状态已改变，但对应after-image无法提交，继续写入会扩大不一致。	立即abort journal；begin_op统一拒绝后续写入并返回EROFS。
强制关闭	管理员要求停止文件系统写入，后台提交继续运行会改变盘面。	shutdown状态单次发布，并按模式完成flush或abort；后续写操作返回EIO。

表3-14 主要健壮性风险及当前处理方式

这些措施不能替代离线e2fsck对既有介质损坏的修复，但能够保证在线内核不会把无法证明安全的状态继续当作正常状态写回。其目标是把错误限制在当前事务或只读文件系统内，避免演化为静默元数据损坏或整个内核崩溃。

实现中为关键状态转换设置了类型限制和调试断言，并通过白盒测试检查这些规则是否得到遵守。例如，Extent叶项编辑函数要求持有EsInvalidated凭证；JBD2提交状态只能沿 Locked、Flush、Commit、CommitRecord、Finished单向推进；缓存命中在调试构建中与权威数据交叉核验。这些机制使违反协议的修改尽可能在编译期或开发测试阶段暴露，而不是等到文件系统镜像损坏后发现。主要约束如表3-15所示。

约束对象	实现约束	验证方式
EsCache	叶项修改前必须取得EsInvalidated；未知范围只能返回Unknown。	调试构建重新遍历ExtentTree 复核coverage。
NodeCache	写回同步刷新槽位，节点释放或复用时清空。	调试构建以新鲜元数据读取逐字节核对。

续下页

续表3-15

约束对象	实现约束	验证方式
JBD2事务阶段	提交状态只允许沿固定后继转换，Finished为终态。	状态转换检查与崩溃注入矩阵。
日志空间	预约credits与revoke块不得超过max_credits，环形空间不足必须先施加backpressure。	tiny-journal并发压力和事务重启测试。
盘上兼容性	元数据写回同步维护特性相关校验和和计数器。	xfstests以及宿主机e2fsck复核。

表3-15 关键实现约束及验证方式

4 性能优化技术研究

4.1 性能评估口径与归因方法

文件系统性能容易受到宿主页缓存、虚拟机启动顺序、构建模式和设备队列状态影响，因此本项目先固定测量口径，再讨论优化效果。Asterinas EXT4与Linux EXT4使用相同的fio参数、文件系统块大小和virtio-blk设备模型；正式结果采用release构建，在同一宿主状态窗口内交替运行两侧负载，并以重复测试的中位数降低偶然波动。SQLite测试在计时结束后执行PRAGMA integrity_check，数据完整性不通过的结果不进入性能统计。

性能归因从文件系统接口、映射与缓存、JBD2事务以及block layer四个层次展开。总耗时首先被分解为文件数据复制、Extent映射、元数据捕获、事务提交、BIO提交和设备等待等部分，再通过journaled、nojournal与裸块设备对照判断瓶颈所属层次。该方法避免把设备波动或缓存命中误认为文件系统优化，也使每项改动都能对应到明确的延迟来源。

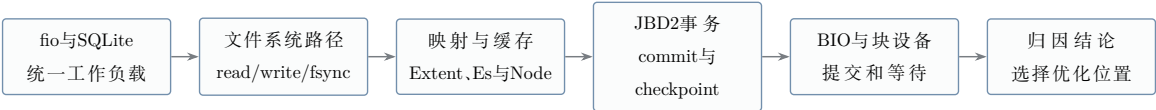


图4-1 性能测量与分层延迟归因流程

4.2 关键瓶颈与优化路线

原生EXT4实现早期的开销分散在树结构修改、空间分配、页写回、重复映射查询和设备请求提交等路径中。项目先通过profiling和文件系统对照确定耗时来源，再针对具体路径修改实现；每次修改后重新运行相关xfstests或崩溃矩阵，并用相同参数比较前后性能。未取得稳定收益或无法通过正确性测试的改动不进入正式实现。

瓶颈	优化方法	主要作用
Extent整树重建	原位插入、分裂、合并和删除	将结构变更从flatten/rebuild收敛为局部节点编辑。
事务额度过度预约	按树深计算credits并支持事务重启	降低日志空间压力，避免大操作一次占满journal。
空间分配局部性不足	块组两档选择、goal推进和inode亲和	减少Extent碎片及跨组元数据访问。
页写回请求过碎	连续页识别、IoBatch与确定性BIO合并	降低逐页提交和完成等待成本。
冷读重复同步等待	滞后扩窗与异步readahead	提前提交后续连续区间，覆盖设备等待时间。
Extent节点重复读取	每inode有界NodeCache	复用Extent节点字节，并以失效和调试复核守住一致性。
映射状态重复证明	EsCache记录已证明状态	在不缓存空洞的前提下减少重复树下降。
inode重复写回	writeback去重与状态薄化	减少同一事务内无新增信息的元数据捕获。
直接I/O固定开销	O_DIRECT分块映射与批量设备I/O	在保持unwritten-first语义的同时扩大单次传输粒度。

高频同步提交开销	掉电通知后的事务回滚与批量持久化	正常运行时暂存完成事务的元数据，收到通知后筛选不完整事务并批量写回。
----------	------------------	------------------------------------

表4-1 主要性能瓶颈与优化路线

4.3 Extent与空间分配优化

原位Extent调整是性能优化的结构基础。旧路径将树展开为内存序列，修改后重新构造节点；其时间、内存和日志额度随整棵树规模增长，在碎片文件、fallocate和数据库索引负载下会形成明显放大。当前实现沿查找路径定位受影响节点，只修改必要的叶节点与索引节点，并在节点满时逐层分裂，在删除后按条件合并或收缩树高。由此，单次结构修改的主要成本由整树规模转为与树深和局部分裂数量相关。

事务预约也随算法同步调整。调用者根据根深和操作类型估算metadata credits，长操作在额度或日志环空间不足时通过可恢复的事务重启继续推进，而不是一次性为最坏情况下的整树重建预留空间。所有原位修改仍先取得journal write access，再写入after-image；性能优化没有绕过Handle、revoke、checksum或checkpoint语义。

空间分配器优先从与inode及当前Extent相邻的块组寻找连续空间，并维护推进中的allocation goal。局部优先策略减少跨块组位图访问和Extent数量，空间紧张时再扩大搜索范围。新分配的数据区间先建立为unwritten extent，数据写入完成后转换为written，保证性能路径仍满足未初始化数据不可见的安全边界。

4.4 缓存与I/O路径优化

PageCache写回先扫描连续脏页，再将物理连续区间组织为IoBatch和较大BIO，减少逐页请求带来的固定开销。读取侧根据顺序访问状态扩展readahead窗口，将下一段设备读取提前提交；随机访问或O_DIRECT不会污染该顺序状态。上述优化只改变请求组织方式，不改变页的脏状态、ordered-data登记和fsync持久化边界。

NodeCache和EsCache分别减少Extent元数据字节读取和映射状态证明。NodeCache是每inode有界缓存，在节点被修改、释放或复用时间同步失效；调试构建可以使用新鲜读取进行逐字节复核。EsCache只保存已经由Extent树证明的written或unwritten等事实，不把一次未命中永久解释为空洞；所有改变映射含义的编辑器必须先取得失效凭证。两类缓存均允许保守未命中退回权威树查询。

O_DIRECT路径按照块对齐范围分块执行映射和BIO提交。直接读先写回重叠脏

页，直接写在一致性窗口内排空并失效重叠缓存页；空洞写使用unwritten-first管线，等待数据I/O成功后再转换映射。表4-2和表4-3中的O_DIRECT结果均在标准JBD2模式下取得；外部电源保护模式使用独立的高频fsync负载评估，不与默认模式结果混合。

4.5 fio性能评估

顺序O_DIRECT测试覆盖4 KiB、16 KiB、64 KiB、256 KiB和1 MiB五种块大小。表4-2列出两侧平均吞吐量及相对比例：Asterinas顺序读达到Linux EXT4的110.0%–113.4%，顺序写达到105.1%–110.4%。随着块大小增大，两侧吞吐量均持续提高；五档测试中Asterinas的读写平均吞吐均高于本轮Linux对照。

测试	Asterinas	Linux EXT4	相对性能
顺序写，4 KiB	41	39	105.1%
顺序读，4 KiB	44	40	110.0%
顺序写，16 KiB	146	137	106.6%
顺序读，16 KiB	161	144	111.8%
顺序写，64 KiB	498	466	106.9%
顺序读，64 KiB	618	552	112.0%
顺序写，256 KiB	712	646	110.2%
顺序读，256 KiB	1898	1705	111.3%
顺序写，1 MiB	743	673	110.4%
顺序读，1 MiB	3750	3308	113.4%

表4-2 fio O_DIRECT顺序读写平均吞吐量与相对性能（MB/s）

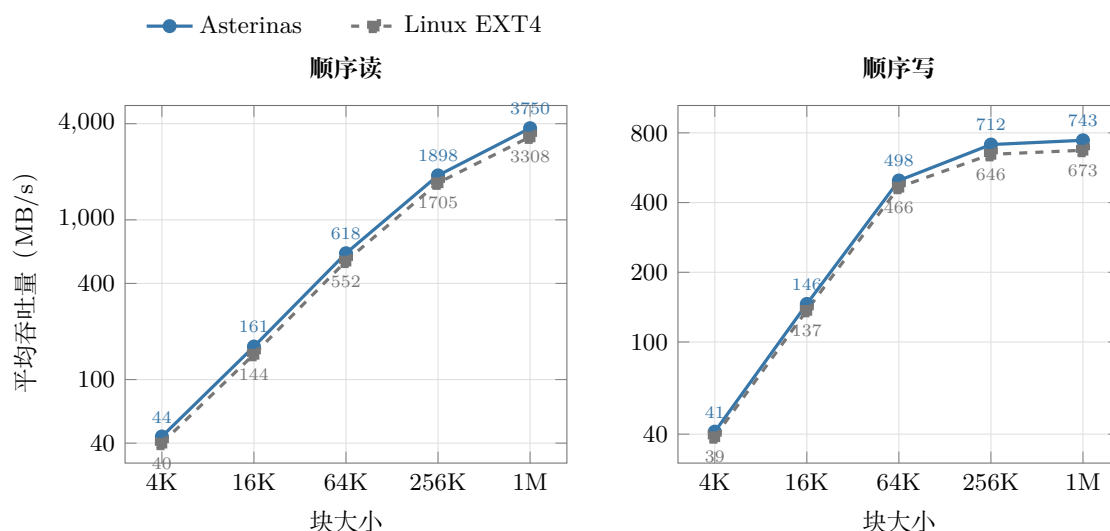


图4-2 fio O_DIRECT顺序读写平均吞吐量对比

同文件并发写使用最新测试结果。numjobs=2时，Asterinas达到1380 MB/s，Linux EXT4为1340 MB/s，相对性能约103%；numjobs=4时分别为2446 MB/s和2194 MB/s，相对性能约111%。该结果用于说明当前写回、映射和设备提交路径能够随并发度扩展。

并发度	Asterinas	Linux EXT4	相对性能
numjobs=2	1380 MB/s	1340 MB/s	103%
numjobs=4	2446 MB/s	2194 MB/s	111%

表4-3 fio同文件并发写性能

4.6 外部电源保护模式性能评估

该模式减少的是正常运行阶段的日志元数据写入、checkpoint和提交屏障，不改变文件数据本身的传输量。普通大块顺序写主要受数据设备吞吐限制，难以充分体现日志路径的差异，因此本测试使用每次写入后执行fsync的顺序写负载，使每个I/O都触发事务同步。测试测量的是日常运行阶段的吞吐率，掉电通知后的冻结、回滚和批量写回不计入fio运行时间。

标准JBD2模式与外部电源保护模式使用同一份Asterinas代码、相同QEMU/KVM环境和相同EXT4镜像参数，仅切换EXT4_POWER_PROTECTED开关。虚拟机配置为单vCPU、8 GiB内存和virtio-blk块设备；每个用例使用新建的2 GiB EXT4镜像，文件系统块大小为4 KiB，并启用journal、extent和metadata checksum等标准特性。各用

例串行运行，测试前执行sync并清理页缓存。

fio采用顺序write、sync ioengine、direct=1、numjobs=1和fsync=1。单个用例写入16 MiB数据，块大小依次为4 KiB、16 KiB、64 KiB、256 KiB和1 MiB。固定总数据量可以保持各用例的数据规模一致；其中4 KiB用例包含4096次同步写入，能够持续触发JBD2提交路径。测试结果见表4-4。

块大小	标准JBD2	外部电源保护	相对性能	性能提升
4 KiB	1.04	2.65	254.81%	154.81%
16 KiB	4.56	10.20	223.68%	123.68%
64 KiB	16.4	39.5	240.85%	140.85%
256 KiB	74.2	134.0	180.59%	80.59%
1 MiB	197.0	357.0	181.22%	81.22%

表4-4 外部电源保护模式下的高频fsync顺序写性能（MB/s）

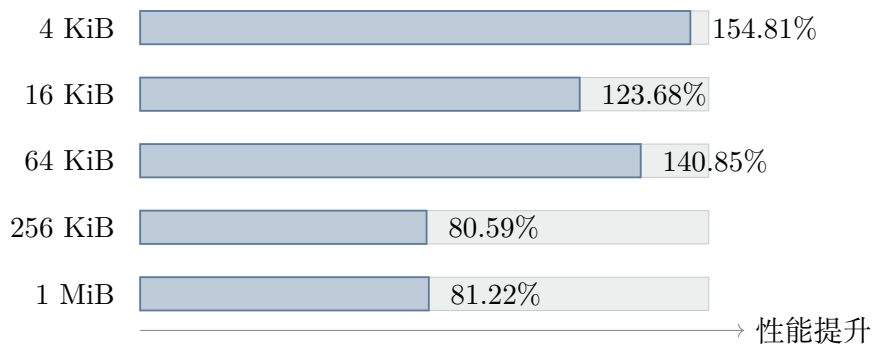


图4-3 外部电源保护模式在高频fsync负载下的性能提升

五种块大小下，外部电源保护模式的吞吐率为标准JBD2模式的180.59%–254.81%，性能提升80.59%–154.81%。4 KiB和64 KiB用例的提升分别达到154.81%和140.85%，说明频繁同步时，descriptor、元数据payload、commit block及相关屏障构成了显著开销。块大小增大后，同一数据量触发的fsync次数减少，但256 KiB和1 MiB用例仍获得约81%的提升。

这组结果量化了正常运行时减少日志I/O的收益，不包含掉电后的处理时间。收到掉电通知后，文件系统仍需关闭操作入口、停止提交线程、丢弃不完整事务，并在UPS供电窗口内写回可信元数据。默认模式继续用于xfstests、崩溃矩阵和Linux互操作验证；只有在掉电通知可靠送达、备用供电时间足够且系统仍能正常执行时，才启用外部电源

保护模式。

4.7 SQLite真实负载评估

SQLite speedtest1同时覆盖文件增长、索引更新、PageCache、目录项变更、journal文件创建删除和高频fsync，比顺序fio更能反映真实应用中的综合成本。本轮对比使用相同的speedtest1工作负载，并将总时间与integrity_check结果绑定。

初赛结果为234.9 s，当前结果为86.2 s，总耗时降低约63.3%，执行速度约为初赛的2.73倍。该比较使用相同工作负载和统计口径。

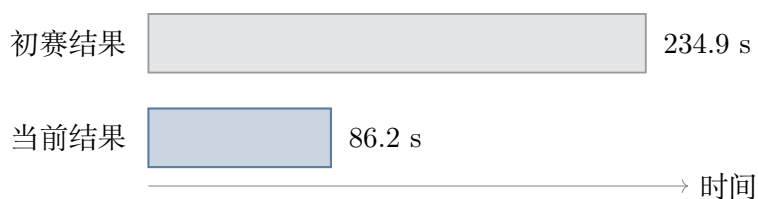


图4-4 SQLite speedtest1两次测试结果对比

86.2 s仍不能说明数据库路径已经追平Linux EXT4。SQLite包含大量小粒度覆盖写和同步事务，当前实现仍需要为多次写入承担元数据捕获、inode更新与提交成本；Linux成熟的delalloc、writeback和事务摊销机制尚未在Asterinas平台上形成同等基础设施。

4.8 RustOS文件系统优化方法总结

本项目形成的优化方法可以概括为三个约束。首先，优化必须以分层profiling和同口径对照为起点，不能根据单次总耗时猜测瓶颈；其次，缓存和树结构优化必须保留明确的权威状态、失效条件和事务捕获凭证；最后，性能结果只有在接口测试、数据完整性检查和崩溃矩阵通过后才具备发布资格。

Rust的所有权、枚举状态和类型化凭证有助于表达这些约束，但不会自动解决日志写序、PageCache一致性或设备流水线问题。原生EXT4的性能提升来自算法、缓存、事务和块层之间的协同；当实验结果不支持某项改动，或者其崩溃安全边界无法闭合时，回退本身也是研究结论的一部分。

5 系统测试与验证

5.1 测试环境与结果

本项目使用QEMU/KVM运行Asterinas，EXT4镜像通过virtio-blk接入，并在相同虚拟化环境中使用Linux EXT4作为接口与性能对照。正式性能数据采用release构建；xfstests使用12 GiB测试与scratch镜像，性能测试按工作负载重新创建镜像，小日志崩溃矩阵将journal限制为4 MiB以主动制造环回绕和空间压力。除第4.6节的专项性能对照外，xfstests、崩溃矩阵和Linux互操作均关闭外部电源保护模式，使用标准JBD2持久化语义。

项目	配置与口径
宿主与虚拟化	Ubuntu宿主，QEMU/KVM，块设备采用virtio-blk。
虚拟机	单处理器、8 GiB内存；需要大scratch空间的xfstests使用12 GiB镜像。
文件系统格式	4 KiB文件系统块，覆盖journal、extent、metadata_csum、64bit和flex_bg等已支持特性。
对照系统	相同工作负载和设备模型下的Linux EXT4，并使用Linux挂载与e2fsck执行互操作检查。
构建口径	正式性能数字仅采用release构建。
证据时间边界	崩溃矩阵采用最终版本；xfstests采用计分子集结果。

表5-1 系统测试环境与结果口径

5.2 测试体系总体设计

测试体系从外部接口逐步扩展到并发访问、崩溃恢复、数据持久化和跨内核磁盘格式互认。xfstests作为Linux文件系统通用回归测试套件，验证VFS和POSIX接口行为[8]；并发与缓存测试覆盖运行时可见性；崩溃矩阵枚举真实块写记录中的持久化前缀；多项独立检查对恢复后的结构、记账、校验和、文件数据和WAL写序分别作出判断；Linux

互操作则检查盘上格式没有形成仅能由本实现读取的私有状态。

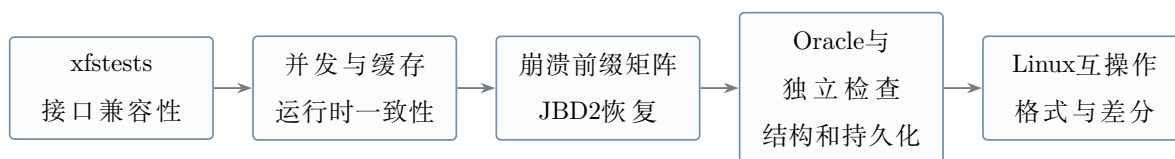


图5-1 从接口行为到跨系统互操作的测试证据链

5.3 xfstests兼容性评估

本次xfstests共计81项：78项PASS、2项稳定FAIL、1项FLAKY，通过率为96.3%。

78项通过用例按主要功能分为六类，每个用例只计入其中一类：

1. 文件与目录命名空间、链接、权限和元数据，共27项。代表用例包括generic/002、generic/035和generic/401，覆盖create/unlink、rename、硬链接、符号链接、目录压力、目录seek、权限与umask、时间戳、mknod、statx及Unicode文件名等行为。
2. 常规读写、截断、追加和数据完整性，共15项。代表用例包括generic/001、generic/014和generic/639，覆盖随机读写校验、fsstress、truncate、洞文件、O_APPEND、向量I/O、splice、高偏移I/O、orphan处理以及卸载重挂后的继续写入。
3. Extent、预分配、fallocate与ENOSPC，共12项。代表用例包括generic/102、generic/213和generic/619，覆盖unwritten Extent、预分配、fallocate范围与对齐、空间预约、满盘重试、并发ENOSPC和块分配一致性。
4. O_DIRECT与buffered/direct一致性，共10项，全部通过。具体用例如下：
generic/130、generic/133、generic/135、generic/214；
generic/406、generic/412、generic/418；
generic/609、generic/615、generic/708。
这些用例覆盖向量直接读写、同文件并发I/O、unwritten Extent直接写、大请求分段、direct/buffered混合访问、PageCache失效、O_DSYNC以及部分直接写。
5. mmap、页缓存一致性与虚拟内存竞争，共13项。代表用例包括generic/030、generic/346和generic/638，覆盖映射写与remap/truncate、mmap与pwrite竞争、prefault、零填充、stale mmap read、PMD/PTE竞争和多页重叠复制。
6. EXT4挂载统计语义，共1项，即ext4/042，用于检查statfs的df/overhead输出以及minixdf、bsddf等挂载选项行为。

稳定失败的generic/127和generic/452均涉及当前mmap实现边界。generic/371在并

发write与fallocate的ENOSPC竞争场景中表现为FLAKY。

5.4 并发与缓存一致性验证

并发测试不能用带宽代替正确性。项目使用确定性数据模式驱动多个worker并发读写，结束后校验每个文件的大小和内容hash；xfstests中的fsstress及并发用例补充随机namespace和空间操作。缓存一致性测试则交叉组合Buffered I/O、O_DIRECT、mmap、truncate与fallocate。

测试族	检查内容	验证方式
多worker文件写入	并发过程中是否出现错写、漏写或文件大小错误。	结束后逐文件核对确定性hash和长度。
Buffered写后直接读	直接读能否观察到尚在PageCache中的先前写入。	重叠范围写回后按字节比较。
直接写后Buffered读	缓存中是否继续保留直接写之前的旧副本。	直接写完成后从普通read路径读回比较。
mmap基本路径	映射页修改与read、write、fsync之间的基本可见性。	读回内容与同步结果比较，并明确平台边界。
namespace与空间压力	rename、unlink、truncate、fallocate和ENOSPC的并发组合。	xfstests、fsstress、无panic及盘后检查。

表5-2 并发与缓存一致性测试设计

5.5 崩溃矩阵与恢复验证

崩溃矩阵不依赖随机终止虚拟机，而是记录一次工作负载产生的块设备写入和FLUSH边界，再对每个可观察持久化前缀构造掉电镜像。每个镜像独立执行JBD2恢复和检查链，能够直接覆盖descriptor、metadata payload、commit block、checkpoint以及日志环回绕之间的写序关系。

工作负载表达与基础语料参考ACE/CrashMonkey提出的J-lang和有界黑盒崩溃测试方法[7]。仓库中的test/crash/jlang2sh.py将可转换的J-lang用例接入Asterinas执行环境，并补充本项目面向JBD2环回绕、revoke复用和多事务提交编写的协议用例。

ACE/CrashMonkey提供工作负载生成思想与部分语料来源；逐FLUSH点记录、Asterinas重启恢复、数据持久化oracle、accounting/checksum检查、walcheck以及结果汇总均由本项目工具链实现。

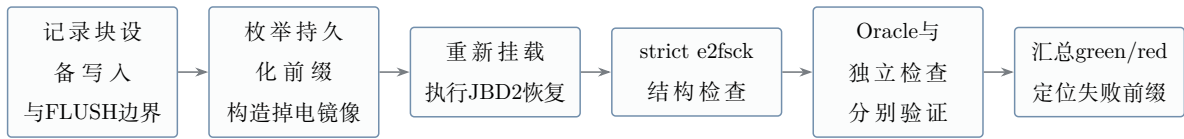


图5-2 块写记录驱动的崩溃矩阵验证流程

在矩阵结果中，green表示对应崩溃前缀通过全部适用检查，red表示至少一项结构、记账、校验和、数据持久化或WAL顺序检查失败。标准日志矩阵覆盖232个工作负载和931个崩溃点，结果为0 red。4 MiB小日志矩阵在相同语料上制造更频繁的journal wrap、checkpoint和空间回收，共验证1629个崩溃点，同样为0 red。两组矩阵合计覆盖2560个持久化前缀；小日志矩阵不是简单重复，而是提高多事务、日志尾推进和checkpoint交错状态的可达性。

矩阵	结果	主要压力
标准日志矩阵	931点，0 red	常规提交、恢复、revoke和checkpoint组合。
4 MiB小日志矩阵	1629点，0 red	日志回绕、空间背压、频繁checkpoint与多事务状态。
合计	2560点，0 red	所有前缀均通过对应恢复与检查链。

表5-3 崩溃矩阵测试结果

5.6 数据持久化Oracle与独立检查体系

仅依靠e2fsck不足以证明fsync数据仍然存在，也无法独立判断空间计数和WAL写序是否正确。项目因此建立互相独立的检查组合：数据持久化oracle记录工作负载中已经建立持久化承诺的文件与字节范围；accounting检查重新计算块和inode记账；checksum检查独立重算盘上metadata_csum；walcheck解析日志事务并检查写回home block的元数据字节是否来自已提交after-image。

检查项	检查问题	测试结果
数据持久化oracle	fsync或fdatasync承诺的数据在任意掉电前缀后是否完整存在。	232个工作负载生效，308条断言全部通过。
strict e2fsck	恢复后的目录、inode、Extent、位图和引用关系是否需要结构修复。	标准与小日志矩阵均为0 red。
accounting检查	空闲块、空闲inode和目录计数是否与重新扫描结果一致。	两类矩阵检查全部通过。
checksum judge	超级块、块组及相关元数据校验和是否与独立重算结果一致。	全部MATCH。
walcheck	home block元数据写入是否具有已提交事务after-image来源。	2360项匹配。

表5-4 崩溃恢复检查及测试结果

数据oracle设置生效性检查，要求预期数量的工作负载和断言实际进入检查，避免因脚本未接线或fsync未执行而得到空绿。

5.7 Linux双向互操作验证

EXT4兼容性最终体现在磁盘镜像能否被不同实现交替使用。项目从两个方向验证互操作：Linux创建和更新的标准EXT4镜像能够由Asterinas挂载、读取并继续修改；Asterinas写入并卸载后的镜像能够由Linux挂载、读取，并通过e2fsck检查。测试同时覆盖metadata_csum、journal checksum v2/v3、64bit journal tag和日志恢复相关格式，防止实现生成只能被自身解析的私有盘面。

方向	验证内容	检查方式
Linux到Asterinas	Linux创建文件系统、目录、文件和带校验和的日志结构。	Asterinas挂载、读取、恢复并继续执行写操作。

Asterinas到Linux	Asterinas执行create、write、fsync、rename、truncate和卸载。	Linux重新挂载后逐项读取，宿主e2fsck检查盘面。
日志格式互认	journal checksum v2/v3、64bit tag、descriptor与commit record。	对照Linux生成字节、恢复结果和独立校验和重算。
差分验证	对相同操作或特定崩溃语义比较两侧可见结果。	区分EXT4标准语义、平台限制与实现缺陷。

表5-5 Linux双向互操作验证

5.8 测试结论与验证边界

xfstests有78项测试通过，其中O_DIRECT相关的10项测试全部通过。崩溃一致性方面，标准日志与4 MiB小日志矩阵共检查2560个崩溃点，所有崩溃点均通过一致性检查；数据持久化oracle覆盖232个工作负载和308条断言，walcheck完成2360个已提交元数据新版本的写序核对。accounting、checksum检查和Linux互操作分别验证空间记账、元数据校验和及磁盘格式兼容性。

上述结论仅适用于已经运行的测试集合、工作负载和崩溃前缀。xfstests中的两项稳定失败对应mmap相关边界，另有一项并发ENOSPC用例存在波动；mmap的部分组合仍受Asterinas VM与PageCache能力限制。

6 项目创新点

本项目的创新不以新增接口数量衡量，而以是否解决了原生文件系统实现中的关键困难为判断标准。围绕Asterinas安全Rust内核、EXT4盘面兼容、JBD2崩溃一致性和高性能I/O路径，项目形成了七项彼此衔接的工程创新，见表6-1。

创新方向	核心设计	可核验证据
安全Rust内核中的原生EXT4	不依赖用户态文件系统或外部EXT4实现，在Asterinas内核内完成VFS接入、盘面结构解析、空间分配、PageCache和块设备I/O，并保持与Linux EXT4的双向盘面互操作。	Linux创建的镜像可由Asterinas读写，Asterinas修改后的镜像可由Linux挂载并通过一致性检查。
完整的原生JBD2生命周期	实现handle与credit、事务切换、ordered data、descriptor与commit block、barrier、checkpoint、revoke以及挂载恢复，使元数据更新从调用入口到恢复重放形成闭环。	两类日志矩阵合计2560个崩溃点0 red；walcheck匹配2360个已提交元数据after-image，0 violation。
原地Extent编辑与深度相关credit	在已有树上完成叶节点编辑、分裂、合并和根提升，避免用整树重建掩盖局部更新成本；事务credit按树高和最坏传播路径估算，使复杂度由文件总Extent数收敛到树深相关范围。	Extent映射检查、满盘边界测试、xfstests与崩溃矩阵共同覆盖映射变化。
带证明边界的语义缓存	EsCache只保存已由树遍历或编辑证明的映射事实，以AllWritten、AllMapped和Unknown表达可用结论；NodeCache保存每inode最近访问的8个外部树节点，并在写回或释放路径维护一致性。	EsInvalidated类型凭证约束叶节点修改前的失效决策；调试构建会用权威树读取交叉检查NodeCache命中。
真实O_DIRECT与缓存一致性协议	直接I/O不借助PageCache模拟：读前写回重叠脏页，写前失效重叠缓存范围；扩展写先建立unwritten映射，数据I/O成功后再转换为written，从而隔离未初始化数据。	O_DIRECT专项xfstests 10项全部通过；同文件并发写在2和4个job下分别达到Linux对照的103%和111%。

续下页

续表6-1

创新方向	核心设计	可核验证据
面向UPS供电窗口的 事务回滚与批量持久化	正常运行时保留ordered data写回和事务状态机，将descriptor、元数据payload、commit block及checkpoint移出高频提交路径；掉电通知后关闭操作入口并停止提交线程，丢弃未完成事务，只把能够确认完整的元数据镜像批量写回。	高频fsync顺序写在五种块大小下提升80.59%–154.81%；回滚与写回实现位于journal/mod.rs，提交分支位于journal/commit.rs。
多项独立检查组成的崩溃验证体系	把写序列记录、逐FLUSH点断电、真实恢复、e2fsck、checksum、accounting、数据持久化oracle和WAL顺序检查组合为互补检查，避免单一“能够挂载”结论漏判。	232个数据持久化工作负载、308条断言通过；标准日志931点和4 MiB小日志1629点均为0 red。

表6-1 项目主要创新点

除外部电源保护模式外，其余优化均保持标准EXT4与JBD2语义。缓存无法命中时重新遍历Extent树，直接I/O失败时不把未完成的数据标记为written，事务完成提交后才允许checkpoint写回最终位置。外部电源保护模式默认关闭；启用后依靠UPS通知和备用供电窗口完成事务筛选与批量持久化，不改变默认配置下的一致性验证结果。

7 开发过程与工程管理

7.1 开发路线

项目按照基础读写、目录操作、JBD2日志与恢复、崩溃测试、数据路径优化和缓存优化的顺序逐步推进。每完成一组功能，先运行对应测试并检查磁盘结果，再继续修改下一条路径。整体开发路线如图7-1所示。

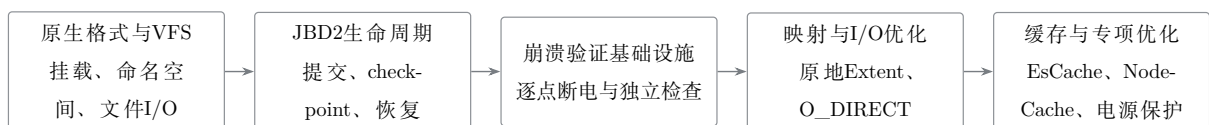


图7-1 项目开发路线

第一阶段以原生EXT4盘面解析、VFS语义和基础文件I/O为目标；第二阶段补齐事

务提交、checkpoint、revoke和挂载恢复；第三阶段建立可枚举崩溃点、可复现镜像和相互独立的检查程序；第四阶段用原地Extent编辑和直接I/O解决映射与数据路径瓶颈；第五阶段完成EsCache、NodeCache和写回优化，并增加带掉电通知回滚的外部电源保护模式及性能对照。各项功能在合入稳定版本前均经过恢复、互操作或一致性检查。

7.2 代码组织与工程约束

当前实现集中在kernel/src/fs/fs_impls/ext4，按文件系统对象、Extent管理、日志、恢复和VFS适配分层。代码组织不仅用于缩短文件长度，更用于固定权威状态和修改入口，见表7-1。

对象	权威状态或入口	工程约束
元数据事务	<code>fs.rs::begin_op</code> 、 <code>OpHandle</code>	所有需要日志保护的修改先申请credit；日志abort后统一返回只读错误。
Extent映射	<code>ExtentTree</code>	EsCache只给出已证明事实；叶节点编辑必须持有EsInvalidated凭证；NodeCache不能成为映射真值来源。
块释放与复用	<code>RevokeDuty</code> 、 <code>BlockFreeAuth</code>	需要revoke的元数据释放必须携带类型凭证，提交持久化后才发布对应revoke表。
持久化等待	<code>sync_tid</code> 、 <code>datasync_tid</code>	<code>fsync</code> 与 <code>fdatasync</code> 等待不同的事务集合，新建inode同时记录两个事务号。
并发访问	全局锁序与inode内部锁	叶级SpinLock不跨设备I/O或事务入口持有；直接写在修改映射前后遵守固定缓存一致性顺序。

表7-1 代码组织与工程约束

缓存无法确认映射状态时，EsCache返回Unknown并重新遍历Extent树；NodeCache中的节点失效后直接清空，后续访问再从设备读取。这样会增加少量读取，但不会使用已经过期的缓存结果。可能改变映射的操作还必须先取得EsInvalidated凭证，从代码接口上固定缓存失效与叶节点修改的先后顺序。

7.3 典型问题与处理

开发中最有价值的问题并非单点语法错误，而是能够暴露错误抽象或验证盲区的问题。表7-2记录了几类具有代表性的现象、定位依据和最终处理。

问题	暴露方式	原因判断	处理结果
Extent更新成本随文件增长	SQLite与碎片化文件中元数据访问放大	整树重建把局部修改退化为与Extent总数相关的工作	改为原地叶编辑和级联分裂，credit按树高与传播路径估算。
新建文件fdatasync遗漏等待	数据持久化oracle构造“创建、写入、fdatasync、断电”序列	新inode只记录sync_tid，datasync_tid为空，导致数据路径错误地认为无需等待	创建事务同时记录两个tid；纯属性更新仍只推进sync_tid。
标准日志WAL检查面过小	崩溃矩阵为绿但walcheck可匹配写入不足	大日志下checkpoint写入不易进入记录窗口，存在“空绿”风险	增加4 MiB小日志矩阵强制wrap/checkpoint，并设置非空匹配门槛。
缓存快路径难以证明	性能优化需要跳过重复树遍历	单纯缓存映射载荷会把失效遗漏升级为错误盘面判断	EsCache改为事实集合和三态结论，引入EsInvalidated；NodeCache命中由调试读取交叉检查。
mmap混合操作边界	truncate、range操作与直接I/O并发测试	VM层缺少完整反向映射和在途页排空，而非EXT4盘面算法缺失	基础mmap保持可用，对无法证明安全的组合显式限制并记录平台依赖。
基准结果波动或失真	多轮fio与SQLite对照出现异常高值	预热、执行顺序、缓存口径和并发基准相互污染	固定镜像与参数、Asterinas/Linux交替运行、采用中位数，并将性能结果绑定完整性检查。

表7-2 典型问题的定位与处理

7.4 代码迭代与测试记录

项目使用Git记录各阶段的代码修改。早期版本先完成挂载、基础读写和目录操作，随后加入JBD2提交与恢复、崩溃测试、O_DIRECT以及Extent和缓存优化。每次功能修改都运行相关的xfstests；涉及日志、空间分配、Extent或写回顺序时，再补充崩溃矩阵检查。未通过测试的实现会继续修改或回退，并在阶段记录中说明原因。

文档中的测试数据均注明所使用的测试集合和运行参数。xfstests、崩溃矩阵和性能测试分别保留运行日志，性能对照还记录镜像配置、fio参数和完整性检查结果。这样可以从文档中的结论找到对应代码版本和测试记录，也能避免把不同配置下的数据直接放在一起比较。

8 AI使用说明

本章按赛事要求披露项目开发过程中AI工具的使用情况，包括工具与模型名称、使用场景、人机分工、AI辅助形成的材料，以及可追溯的交互记录。

赛题：2026全国大学生计算机系统能力大赛·操作系统设计赛·OS功能挑战赛道。

项目：在Asterinas（Rust framekernel）上用Rust实现兼容POSIX、支持Extent与JBD2日志机制的EXT4文件系统，并探索面向RustOS架构的性能优化技术。

8.1 披露范围

本项目使用AI的基本方式是：参赛队负责需求拆分、功能实现、代码修改、测试取舍、结果验收和最终提交；AI工具主要用于沟通实现思路、检查代码风险、排查运行bug、生成自动化测试脚本、整理测试数据表格，以及总结阶段状态供队内讨论。

8.2 AI工具与模型清单

项	内容
交互与执行工具	Claude Code（命令行交互与执行harness）
使用的大模型	DeepSeek V4 Pro
使用周期	2026-07至2026-08
使用方式	本地终端交互

8.3 使用场景

AI工具主要用于以下辅助场景：

- **思路沟通与代码检查**：围绕参赛队已经实现或准备实现的功能，讨论可能影响的模块、边界条件和风险点，用作人工检查清单。
- **运行问题排查**：当功能运行不正确、测试失败或出现panic/timeout时，辅助分析日志和报错信息，缩小bug排查范围。
- **自动化测试辅助**：由于xfstests、crash recovery、SQLite、fio等测试耗时较长，AI辅助编写和维护测试脚本、整理运行命令，并按参赛队要求执行重复测试。
- **测试数据整理**：将多轮benchmark、profile、PASS/FAIL结果整理为表格，便于比较不同阶段的性能变化和正确性状态。
- **阶段状态总结**：根据已运行的测试结果和当前代码状态，总结阶段完成情况、遗留问题和下一步计划，方便队内沟通。

8.4 人机分工

参赛队负责全部核心决策与实现工作：确定阶段目标和优先级，例如JBD2日志、PageCache、O_DIRECT、SQLite写性能等开发顺序；完成功能实现和代码修改，包括extent映射、事务边界、dirty page写回、fsync/fdatasync、并发锁和crash recovery相关逻辑；审查代码改动是否符合项目设计；并决定最终提交内容，维护Git分支和commit记录。

AI工具仅承担辅助性工作：根据报错日志、测试输出和运行现象辅助定位问题；编写或调整测试脚本，帮助重复运行耗时测试；把多轮测试数据整理成表格或阶段总结，便于队伍判断下一步方向；并对部分关键路径提供代码安全检查，供参赛队判断是否采纳。

8.5 交互记录与可追溯性

与AI的交互记录以文档的形式保存在docs/AI_usage/。这些文档记录了各阶段的测试结果分析和阶段总结。

8.6 诚信声明

本文件如实披露了项目中使用的AI工具、模型名称和主要使用场景。AI参与的是思路沟通、代码问题检查、bug排查、自动化测试脚本、测试运行辅助和数据表格整理

等工作；参赛队负责功能实现、核心设计、代码审查、测试验收和最终提交。

9 总结

本项目在Asterinas安全Rust内核中实现了原生EXT4文件系统。系统直接读写标准EXT4盘面，完成VFS、Extent、空间分配、PageCache、Buffered I/O、O_DIRECT、fsync/fdatasync和基础mmap路径，并能够与Linux进行双向镜像互操作。磁盘结构解析、元数据更新和一致性维护均由内核中的Rust代码完成，运行过程不依赖外部文件系统转发。

围绕崩溃一致性，项目实现了完整的JBD2事务生命周期：元数据修改通过OpHandle和credit进入事务，提交线程依次处理ordered data、日志写入、barrier和commit record，checkpoint在事务持久后更新home block；revoke在提交持久后发布并在恢复扫描中阻止旧元数据重放。fsync与fdatasync分别等待sync_tid和datasync_tid，使持久化边界与操作语义对应。

围绕性能，项目以原地Extent编辑替代整树重建，以EsCache保存可证明的映射事实，以NodeCache消除热树节点的重复读取，并为直接I/O建立页缓存写回与失效协议。五档O_DIRECT顺序读达到Linux EXT4的110.0%–113.4%，顺序写达到105.1%–110.4%；同文件并发写在2个job下达到1380 MB/s，在4个job下达到2446 MB/s，分别为Linux对照的103%和111%。SQLite speedtest1由234.9秒降至86.2秒。

针对配有UPS、冗余供电或电池备份单元的环境，项目进一步实现了可选的外部电源保护模式。正常运行时，文件数据仍按ordered语义写入设备，完成事务的元数据版本暂存于内存，从而减少日志写入和checkpoint开销。收到掉电通知后，文件系统关闭新的修改入口并停止提交线程，丢弃Running、Locking以及无法确认完成的Committing事务，只将Finished事务对应的可信元数据批量写回home block，执行flush和barrier后安全关机。高频fsync顺序写在五种块大小下达到标准JBD2模式的180.59%–254.81%，性能提升80.59%–154.81%。该模式依赖可靠的掉电通知和约3–5分钟的备用供电窗口，标准JBD2仍为默认配置。

验证体系覆盖xfstests、O_DIRECT专项、逐点崩溃、数据持久化oracle、checksum/accounting、walcheck和Linux互操作。xfstests有78项测试通过，两类日志矩阵共检查2560个崩溃点并全部通过一致性检查。多项检查分别覆盖结构、引用计数、校验和、应用可见数据和写前日志关系，降低了单一测试结果造成误判的可能。

本项目说明，使用Safe Rust构建复杂的原生磁盘文件系统，不仅能够获得内存安全方面的保障，也能够兼顾Linux磁盘格式兼容、事务一致性和高性能数据访问。项目

将Extent管理、JBD2日志、缓存协同、直接I/O和系统化验证贯通为一套完整方案，为Asterinas运行更丰富的存储型应用奠定了基础，也为Rust操作系统实现具有持久状态和复杂并发关系的内核子系统提供了可复用的设计经验。

附录 A 核心实现细节补充

A.1 事务入口与OpHandle

fs.rs中的begin_op是文件系统元数据事务的统一入口。它先检查文件系统是否shutdown或只读，再检查journal是否abort，最后根据挂载配置返回真实OpHandle或无日志句柄。当前源码如下：

```

1 pub(super) fn begin_op(&self, credits: usize)
2     -> Result<journal::OpHandle>
3 {
4     self.ensure_not_shutdown()?;
5     self.ensure_writable()?;
6     match self.journal() {
7         Some(journal) => {
8             if journal.is_aborted() {
9                 return_errno_with_message!(
10                     Errno::EROFS,
11                     "filesystem is read-only after a journal abort"
12                 );
13             }
14             journal::OpHandle::start(&journal, credits)
15         }
16         None => Ok(journal::OpHandle::none()),
17     }
18 }
```

OpHandle以RAII方式约束一次高层操作持有的journal handle，并把credit上界传给JBD2。目录修改、inode更新、空间分配和Extent编辑都通过该入口建立事务上下文，因而错误传播、只读降级和事务等待具有统一语义。代码位置为kernel/src/fs/fs_impls/ext4/fs.rs。

A.2 EsCache事实语义

EsCache只保存Extent树遍历或编辑时已经确认的映射事实，不作为完整映射表，也不记录空洞。未记录区域返回Unknown并重新遍历Extent树。其查询结论见表A-1。

结论	已证明事实	允许的使用方式
AllWritten	请求范围全部由written Extent覆盖	允许进入纯覆盖写快速路径。
AllMapped	全部已映射，但至少包含unwritten	可证明无需填洞，不能当作有效旧数据。
Unknown	至少一个块没有缓存事实	必须遍历Extent树，不能推断为空洞。

表A-1 EsCache三态结论

缓存最多保存每inode 512项；达到容量时整体清空后重新积累。删除事实只会产生额外树遍历，因此整体清空不影响正确性。所有可能改写叶条目的路径先调用范围失效或全量失效并取得不可伪造的EsInvalidated凭证，叶编辑函数要求该凭证，从类型层面固定“先失效、后修改”的顺序。代码位置为inode/extent_manager/es.rs。

A.3 NodeCache一致性规则

NodeCache位于每个ExtentTree内部，保存最近访问的8个外部Extent树节点，每项为节点物理块号及4 KiB字节副本。相同块写回时更新对应slot，空slot耗尽后按round-robin替换；外部节点释放或整树重建时保守清空。它只缩短节点读取，不参与决定Extent语义。

事件	缓存动作	一致性依据
节点读取	命中时克隆字节，未命中走权威 读取并填充	slot锁在设备I/O前释放。
节点写回	用最终字节更新同一物理块slot	缓存内容与NodeBuf写回结果同步。
节点释放或重建	清空全部slot	保守失效只损失命中率。
调试构建命中	额外执行权威读取并比较	调试断言持续检查缓存字节一致性。

表A-2 NodeCache一致性规则

代码位置为inode/extent_manager/tree.rs。内部SpinLock只保护slot探测与替换，不跨设备I/O、journal入口或其他锁持有，因此不增加ExtentTree锁之外的睡眠锁序边。

A.4 revoke正确性

revoke不是恢复侧的孤立过滤器，而是贯穿运行事务、提交和恢复的协议。运行事务收集被撤销的元数据块；提交线程将revoke记录写入日志，并且只有在commit持久后才把集合发布到已提交RevokeTable；恢复的PASS_REVOKE先扫描记录，再在重放after-image时按事务序列过滤。

块释放接口使用BlockFreeAuth区分数据块和元数据块。释放可能被旧事务重放覆盖的元数据块时，调用者必须提供RevokeDuty类型凭证；普通数据块可以明确选择无revoke义务的授权。该设计把“此处是否必须revoke”的判断保留在调用关系和类型检查中，避免依赖注释约定。对应实现位于journal/revoke.rs及空间释放调用路径。

A.5 fsync与fdatasync事务号

每个inode分别记录sync_tid和datasync_tid。前者表示完整fsync需要等待的最新事务，后者表示fdatasync为了数据和读取数据所需元数据必须等待的子集，见表A-3。

操作	sync_tid	datasync_tid	原因
新建inode	推进	推进	文件的存在是其数据可见性的前提。
数据或大小变化	推进	推进	映射、大小与数据读取直接相关。
纯属性变化	推进	不推进	fsync需等待，fdatasync无需强制仅属性事务。

表A-3 inode事务号更新规则

fsync在脏页写回后等待sync_tid，fdatasync等待datasync_tid。新建inode同时记录两个事务号，修复了“创建后写入并fdatasync，但等待目标为空”的持久化漏洞。对应实现位于inode/mod.rs和impl_for_vfs/inode.rs。

附录 B 复现命令

B.1 复现入口

测试需在项目锁定的开发容器内运行，部分端到端测试还要求Linux宿主提供KVM、loop设备权限和足够的临时磁盘空间。容器入口由.devcontainer/devcontainer.json、tools/docker/Dockerfile和tools/docker/run_dev_container.sh共同固定。

类别	仓库入口	输出或用途
静态检查	make check	执行格式、Clippy、拼写与许可证检查。
xfstests	test/initramfs/src/conformance/xfstests/full.list	guest内由run_xfstests.sh调用官方check并保存计分用例的完整结果。
崩溃矩阵	test/crash/run_matrix.sh	转换工作负载、记录写序列，并对每个FLUSH前缀执行恢复和多项独立检查。

续表B-1

类别	仓库入口	输出或用途
数据oracle校准	test/crash/oracle_selfcheck.sh	证明oracle能够放行正确镜像，并能对删除已fsync文件或染色数据复活主动报红。
WAL顺序检查	test/crash/walcheck.py	核对已提交元数据版本与写回顺序，并由-walcheck门槛确认检查实际执行。
性能测试	test/bench/README.md及同目录跑批、解析脚本	使用相同参数分别运行Asterinas和Linux对照测试，并保存每次运行的原始日志与汇总结果。
外部电源保护模式	make run_kernel EXT4_POWER_PROTECTED=1	通过内核参数 ext4.power_protected=1启用掉电通知回滚与批量持久化；默认值为0。

表B-1 主要复现入口

静态检查和xfstests计分清单可按以下形式运行：

```

1 cd os_com-rebuild11
2 git rev-parse HEAD
3 make check
4 make run_kernel AUTO_TEST=conformance \
5     CONFORMANCE_TEST_SUITE=xfstests \
6     XFSTESTS_RUNLIST=/opt/xfstests/full.list \
7     XFSTESTS_DISK_SIZE=12G MEM=8G RELEASE=1

```

外部电源保护模式与标准JBD2模式使用同一代码版本，性能对照仅切换以下运行参数：

```

1 make run_kernel EXT4_POWER_PROTECTED=0
2 make run_kernel EXT4_POWER_PROTECTED=1

```

上述命令用于比较正常运行阶段的高频fsync性能。启用外部电源保护模式后，UPS掉电通知通过EXT4_IOC_SHUTDOWN进入文件系统关闭路径，依次完成入口冻结、提交线程停止、不完整事务丢弃、可信元数据写回和设备屏障；普通受控卸载也会持久化已经完成的内存事务。

崩溃矩阵的基本调用形式如下。标准日志与4 MiB小日志必须使用同一份固定语料；其中<jlang-corpus-dir>、工作负载名单及其校验和由对应证据清单给出，不能临时替换：

```
1 bash test/crash/run_matrix.sh [--shards N] --keep \  
2     <jlang-corpus-dir> [workload-name ...]  
3  
4 bash test/crash/run_matrix.sh [--shards N] --keep \  
5     --journal-size 4 --walcheck <minimum-matches> \  
6     <jlang-corpus-dir> [workload-name ...]
```

4 MiB小日志验证通过-journal-size 4启用，walcheck通过-walcheck N设置最小非空匹配门槛。run_matrix.sh在test/initramfs/build/下生成matrix.sK.progress.tsv、matrix.sK.sweep.log、matrix.sK.oracle.table、matrix.sK.walcheck.log、matrix.sK.final.manifest和matrix.peaks等产物；出现red时还保留写日志、pristine镜像及matrix-evidence/复现目录。

性能测试以test/bench/README.md所述host侧跑批入口为准。正式对照固定release构建、8 GiB内存、单vCPU、2 GiB测试盘、EXT4全特性镜像，并在每个case前重建镜像和清理宿主缓存；fio使用1 GiB文件、sync ioengine及表4-2和表4-3中的块大小与numjobs，SQLite执行sqlite-speedtest1 -size 1000并在计时后运行PRAGMA integrity_check。外部电源保护专项按照第4.6节的16 MiB高频fsync负载执行，并分别记录开关为0和1时的结果。测试结果记录对应commit、运行参数、测量值和完整性检查状态，原始启动日志按测试批次分别保存。

参考资料

- [1] Avantika Mathur, Mingming Cao, Suparna Bhattacharya, Andreas Dilger, Alex Tomas, and Laurent Vivier. *The New EXT4 Filesystem: Current Status and Future Plans*. Proceedings of the Linux Symposium, Vol. 2, pp. 21–34, 2007. <https://www.kernel.org/doc/ols/2007/ols2007v2-pages-21-34.pdf>.
- [2] Linux Kernel Community. *EXT4 General Information*. Linux Kernel Documentation. <https://docs.kernel.org/admin-guide/ext4.html>.
- [3] Linux Kernel Community. *EXT4 Block and Inode Allocation Policy*. Linux Kernel Documentation. <https://docs.kernel.org/filesystems/ext4/allocators.html>.
- [4] Linux Kernel Community. *Overview of the Linux Virtual File System*. Linux Kernel Documentation. <https://docs.kernel.org/filesystems/vfs.html>.
- [5] Linux Kernel Community. *Linux EXT4 Source Code*. <https://elixir.bootlin.com/linux/latest/source/fs/ext4>.
- [6] Lanyue Lu, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau, and Shan Lu. *A Study of Linux File System Evolution*. In Proceedings of the 11th USENIX Conference on File and Storage Technologies (FAST '13), pp. 31–44, 2013. <https://research.cs.wisc.edu/adsl/Publications/fsstudy-fast13.pdf>.
- [7] Jayashree Mohan, Ashlie Martinez, Soujanya Ponnappalli, Pandian Raju, and Vijay Chidambaram. *Finding Crash-Consistency Bugs with Bounded Black-Box Crash Testing*. In Proceedings of the 13th USENIX Symposium on Operating Systems Design and Implementation (OSDI '18), pp. 33–50, 2018. <https://www.usenix.org/conference/osdi18/presentation/mohan>.
- [8] xfstests Developers. *xfstests: File System Regression Test Suite*. <https://git.kernel.org/pub/scm/fs/xfs/xfstests-dev.git>.
- [9] Samantha Miller, Kaiyuan Zhang, Mengqi Chen, Ryan Jennings, Ang Chen, Danyang Zhuo, and Thomas Anderson. *High Velocity Kernel File Systems with*

- Bento*. In Proceedings of the 19th USENIX Conference on File and Storage Technologies (FAST '21), pp. 65–79, 2021. <https://www.usenix.org/conference/fast21/presentation/miller>.
- [10] Hayley LeBlanc, Nathan Taylor, James Bornholt, and Vijay Chidambaram. *SquirrelFS: Using the Rust Compiler to Check File-System Crash Consistency*. In Proceedings of the 18th USENIX Symposium on Operating Systems Design and Implementation (OSDI '24), pp. 387–404, 2024. <https://www.usenix.org/conference/osdi24/presentation/leblanc>.
- [11] Kevin Boos, Namitha Liyanage, Ramla Ijaz, and Lin Zhong. *Theseus: An Experiment in Operating System Structure and State Management*. In Proceedings of the 14th USENIX Symposium on Operating Systems Design and Implementation (OSDI '20), pp. 1–19, 2020. <https://www.usenix.org/conference/osdi20/presentation/boos>.
- [12] Yuke Peng, Hongliang Tian, Junyang Zhang, Ruihan Li, Chengjun Chen, Jianfeng Jiang, Jinyi Xian, Xiaolin Wang, Chenren Xu, Diyu Zhou, Yingwei Luo, Shoumeng Yan, and Yinqian Zhang. *ASTERINAS: A Linux ABI-Compatible, Rust-Based Framekernel OS with a Small and Sound TCB*. In Proceedings of the 2025 USENIX Annual Technical Conference (USENIX ATC '25), pp. 307–323, 2025. <https://www.usenix.org/conference/atc25/presentation/peng-yuke>.
- [13] Asterinas Team. *Asterinas GitHub Repository*. <https://github.com/asterinas/asterinas>.
- [14] yuoo655. *ext4_rs: An OS-Independent Rust EXT4 File System*. https://github.com/yuoo655/ext4_rs.
- [15] Christina Peabody and Gregory Sizikov. *How We Got to 100 Million Cells in Our Global Li-ion Rack Battery Fleet*. Google Cloud Blog, 2025. <https://cloud.google.com/blog/topics/systems/100-million-li-ion-cells-in-google-data-centers>.
- [16] Open Compute Project. *Open Rack V3 BBU Module Specification, Version 1.4*. 2026. <https://www.opencompute.org/documents/open-rack-v3-bbu-module-spec-1-4-pdf>.

- [17] Microsoft. *Datacenter Architecture and Infrastructure*. Microsoft Service Assurance.
<https://learn.microsoft.com/compliance/assurance/assurance-datacenter-architecture-infrastructure>.